

Transformers and LLMs - Interview Guide

This guide covers the core Transformer and LLM concepts asked in placements, AI engineer interviews, research internships, and project discussions. Each topic follows the same study structure, with practical formulas, code, interview angles, and revision notes.

Tokenization

1. Overview

Tokenization converts raw text into smaller units called tokens. Tokens may be characters, words, subwords, bytes, or special symbols. It is useful because neural networks cannot directly process strings; they need integer IDs that can be mapped to embeddings. Tokenization is used in every NLP and LLM pipeline: search, chatbots, translation, summarization, RAG, sentiment analysis, and code generation.

2. Intuition

Think of tokenization as cutting a sentence into model-readable pieces. For example, "unhappiness" may become ["un", "happiness"] or ["un", "happy", "ness"]. Subword tokenization helps models understand rare words by composing them from known pieces.

3. Prerequisites

- Python strings and Unicode basics
- Vocabulary, integer encoding, and one-hot vectors
- Basic NLP preprocessing
- Probability and frequency counts
- Transformer input pipelines

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Vocabulary	Set of known tokens	Defines representable text	["the", "cat", "##s"]	Vocabulary size vs memory
Token ID	Integer assigned to token	Model input is integer IDs	"cat" -> 3457	IDs are not semantic by themselves
Special tokens	Control tokens	Mark padding, BOS, EOS, masks	[PAD], <eos>	Padding masks and generation stop
Subword tokens	Pieces of words	Handles rare/OOV words	"playing" -> "play", "ing"	BPE/WordPiece/Unigram

Concept	Meaning	Why it matters	Example	Interview angle
Byte tokens	Byte-level pieces	Robust to any text	GPT-style byte BPE	Unicode handling

5. Algorithm / Working Process

1. Normalize text if the tokenizer requires it.
2. Split text into candidate units such as bytes, characters, or words.
3. Apply a learned segmentation algorithm such as BPE, WordPiece, or Unigram.
4. Map each token to an integer ID from the vocabulary.
5. Add special tokens if required.
6. Pad or truncate to the model's maximum length.
7. Return `input_ids` and usually `attention_mask`.

Input: raw text.

Output: token IDs and masks.

Training process: learn vocabulary/merge rules from a corpus.

Inference process: apply fixed tokenizer rules learned during training.

6. Mathematical Foundation

Tokenization is mostly algorithmic, but training uses frequency and likelihood.

BPE starts from characters/bytes and repeatedly merges the most frequent pair:

```
merge* = argmax_(a,b) count(a,b)
```

Unigram tokenization chooses a segmentation `S` that maximizes likelihood:

```
P(text) = product over token t in S of P(t)
best_S = argmax_S P(text)
```

Important trade-off:

```
larger vocabulary -> shorter sequences but larger embedding table
smaller vocabulary -> longer sequences but better rare-word coverage
```

7. Practical Implementation

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

```
texts = [
    "Transformers changed modern NLP.",
    "Tokenization handles unseen words using subwords."
]

batch = tokenizer(
    texts,
    padding=True,
    truncation=True,
    max_length=16,
    return_tensors="pt"
)

print(batch["input_ids"])
print(batch["attention_mask"])
print(tokenizer.convert_ids_to_tokens(batch["input_ids"][0]))
```

8. Code Explanation

`AutoTokenizer` loads the exact tokenizer used by the pretrained model. `padding=True` makes all examples in the batch the same length. `truncation=True` prevents inputs from exceeding the model limit. `attention_mask` marks real tokens with `1` and padding with `0`.

9. Training / Evaluation

For tokenizer training, use a representative corpus. Evaluate average tokens per word, unknown-token rate, compression ratio, domain coverage, and downstream task performance. Poor tokenization can hurt domain tasks such as medical NLP, Indian-language NLP, legal search, and code models.

10. Complexity and Cost

Tokenization is usually CPU-bound. Runtime is roughly linear in text length, though algorithms differ. Memory cost is mainly vocabulary size and embedding table size:

```
embedding_parameters = vocabulary_size * embedding_dim
```

11. Common Use Cases

- Preparing text for BERT, GPT, T5, LLaMA, and other LLMs
- RAG chunk counting
- Chatbot prompt budgeting
- Search indexing
- Code model preprocessing

12. Common Mistakes

- Using a tokenizer different from the model's tokenizer
- Counting words instead of tokens for context limits

- Forgetting padding masks
- Truncating away labels or key context
- Training a tokenizer on a corpus that does not match the target domain

13. Edge Cases / Limitations

Tokenizers may split names, URLs, emojis, code, math, and low-resource-language text poorly. Tokenization can also introduce bias because some languages require more tokens for the same meaning, increasing cost and reducing effective context.

14. Variations

Variation	What changes	When to use	Placement value
Word tokenization	Splits by words	Classical NLP	Basic
Character tokenization	Uses characters	No OOV issue	Good for spelling/noisy text
BPE	Frequent pair merges	GPT-style models	Very important
WordPiece	Likelihood-inspired subwords	BERT-style models	Very important
SentencePiece/Unigram	Language-independent segmentation	Multilingual models	Important
Byte-level BPE	Works on bytes	Robust LLM tokenization	Important

15. Related Topics

Tokenization connects to embeddings because token IDs index embedding vectors. It connects to context window because token count, not word count, determines length. It connects to prompting because concise prompts save tokens. It connects to RAG because chunk sizes are usually token-based.

16. Interview Questions

1. What is tokenization?

It converts text into model-readable tokens and token IDs.

2. Why do LLMs use subword tokenization?

It handles rare words while keeping vocabulary manageable.

3. What is an OOV token?

An out-of-vocabulary token that the tokenizer cannot represent directly.

4. Why must the tokenizer match the model?

Token IDs map to learned embeddings; mismatched IDs mean wrong inputs.

5. What does `attention_mask` do?

It prevents the model from attending to padding tokens.

6. BPE vs WordPiece?

BPE merges frequent pairs; WordPiece chooses merges with a likelihood-style criterion.

7. Why are emojis and code difficult?

They may split into many tokens and waste context.

8. What happens if text is truncated?

The model loses information beyond the maximum length.

9. How does vocabulary size affect cost?

Larger vocabularies increase embedding and output projection parameters.

10. Why does tokenization matter in RAG?

Chunk size and prompt length must fit within the context window.

17. Practice Tasks

- Tokenize the same paragraph with BERT and GPT-2 tokenizers and compare token counts.
- Train a tiny BPE tokenizer on a custom corpus.
- Debug a classification model where labels are truncated accidentally.
- Measure token cost for different prompt templates.
- Compare tokenization quality for English, Hindi, code, and URLs.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Token Cost Analyzer	Estimates LLM prompt cost	Python, HF tokenizers	User prompts	Useful AI engineering tool
Domain Tokenizer	Trains tokenizer for medical/legal text	SentencePiece	PubMed/legal docs	Shows NLP preprocessing depth
Multilingual Token Audit	Compares token efficiency by language	Python, pandas	FLORES/Wikipedia	Good fairness/LLM analysis

19. Quick Revision

- Key idea: text must become token IDs.
- Main formula: embedding parameters = $V * d$.
- When to use: every NLP/LLM system.
- Important metrics: token count, OOV rate, compression ratio.
- Common traps: wrong tokenizer, truncation, ignored padding mask.
- Interview one-liner: tokenization is the bridge between raw language and neural input IDs.

20. Final Cheat Sheet

Item	Summary
Definition	Splitting text into model-readable units
Input/output	Text -> token IDs + masks
Main steps	normalize, split, segment, map, pad/truncate
Hyperparameters	vocab size, max length, special tokens

Item	Summary
Metrics	token count, OOV rate, downstream score
Pros	handles text efficiently
Cons	language/domain bias, token inflation
Best use cases	all NLP and LLM pipelines

Embeddings

1. Overview

Embeddings are dense vector representations of discrete items such as tokens, words, sentences, documents, users, products, or images. In LLMs, token embeddings convert token IDs into continuous vectors that can be processed by neural networks. Embeddings are used in language models, recommendation systems, semantic search, RAG, clustering, anomaly detection, and multimodal AI.

2. Intuition

An embedding places similar items near each other in a vector space. If "king" and "queen" appear in similar contexts, their vectors become close. Instead of treating words as unrelated IDs, embeddings let the model learn relationships.

3. Prerequisites

- Vectors, matrices, dot product, cosine similarity
- Neural network training and backpropagation
- Tokenization
- Loss functions
- Basic PyTorch tensors

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Embedding matrix	Lookup table $V \times d$	Maps token IDs to vectors	ID 42 -> row 42	Parameters learned during training
Dimensionality	Vector length d	Controls capacity and cost	768, 4096	Larger is not always better
Similarity	Vector closeness	Enables semantic search	cosine score	Dot product vs cosine
Contextual embedding	Depends on sentence	Same word can mean different things	"bank" river/finance	Static vs contextual

Concept	Meaning	Why it matters	Example	Interview angle
Sentence embedding	Whole text vector	Retrieval and clustering	document vector	Pooling strategy matters

5. Algorithm / Working Process

Input: token IDs $[t_1, t_2, \dots, t_n]$.

Processing:

1. Initialize or load embedding matrix E .
2. For each token ID t_i , select row $E[t_i]$.
3. Add positional information if used.
4. Pass vectors into Transformer layers.
5. Update embedding rows during training through backpropagation.

Output: matrix of shape $(\text{sequence_length}, \text{embedding_dim})$.

6. Mathematical Foundation

For vocabulary size V and embedding dimension d :

$$E \in \mathbb{R}^{(V \times d)}$$

$$x_i = E[\text{token_id}_i]$$

Cosine similarity:

$$\cos(a, b) = (a \cdot b) / (||a|| \ ||b||)$$

In language modeling, embeddings are learned to reduce prediction loss:

$$L = - \sum_t \log P(\text{token}_t \mid \text{context})$$

Many LLMs tie input embedding and output projection weights:

$$\text{logits} = h_t E^T$$

7. Practical Implementation

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
vocab_size = 1000
embedding_dim = 64

embedding = nn.Embedding(vocab_size, embedding_dim)
token_ids = torch.tensor([[10, 45, 932, 7]])

x = embedding(token_ids)
print(x.shape)  # [batch, seq_len, embedding_dim]

# Similarity between two token vectors
sim = F.cosine_similarity(x[:, 0, :], x[:, 1, :])
print(sim)
```

8. Code Explanation

`nn.Embedding` creates a learnable lookup table. Passing token IDs returns vectors. The output shape includes batch size, sequence length, and embedding dimension. `cosine_similarity` measures whether two vectors point in similar directions.

9. Training / Evaluation

Embeddings are trained with the task model. Evaluate them through downstream metrics such as accuracy, F1, retrieval recall, MRR, nDCG, clustering quality, or semantic similarity correlation. Good embeddings should generalize to unseen examples, not just memorize token IDs.

10. Complexity and Cost

Embedding parameters:

$$V * d$$

Lookup cost is cheap, roughly $O(\text{sequence_length})$, but memory can be large for big vocabularies and large dimensions. Output softmax over vocabulary is often expensive in language models.

11. Common Use Cases

- Token representation in Transformers
- Semantic document retrieval in RAG
- Recommendation systems
- Similar question detection
- Image-text retrieval
- Clustering and visualization

12. Common Mistakes

- Comparing raw unnormalized embeddings with dot product when cosine is intended
- Assuming all embeddings are contextual

- Using poor pooling for sentence embeddings
- Mixing embedding models across index and query
- Ignoring embedding drift after fine-tuning

13. Edge Cases / Limitations

Embeddings can encode bias from data. Similarity can fail for negation, numbers, exact constraints, and rare entities. A dense embedding may retrieve semantically similar but factually wrong documents.

14. Variations

Variation	What changes	When to use	Placement value
One-hot	Sparse vector	Teaching baseline	Basic
Word2Vec/GloVe	Static word vectors	Classical NLP	Important historically
Contextual embeddings	Depend on context	BERT/LLMs	Very important
Sentence embeddings	Whole-text vectors	RAG/search	Very important
Multimodal embeddings	Text/image/audio same space	CLIP-style tasks	Research/project value

15. Related Topics

Embeddings connect to tokenization through token IDs. They connect to attention because attention consumes embedded vectors. They connect to RAG because retrieval depends heavily on embedding quality. They connect to contrastive learning because many embedding models are trained by pulling positives together and pushing negatives apart.

16. Interview Questions

1. What is an embedding?
A dense learned vector representation of a discrete item.
2. Why not use one-hot vectors directly?
One-hot vectors are sparse, high-dimensional, and do not encode similarity.
3. What is embedding dimension?
The number of values in each vector.
4. How are embeddings learned?
Through gradient descent from a task loss.
5. What is cosine similarity?
A normalized dot product measuring direction similarity.
6. Static vs contextual embeddings?
Static vectors are fixed per word; contextual vectors depend on surrounding text.
7. Why are embeddings useful in RAG?
They allow semantic search over documents.
8. What is weight tying?
Sharing input embedding weights with output vocabulary projection.
9. What happens with unseen tokens?
The tokenizer decomposes them or maps to unknown tokens.

10. Can embeddings be biased?

Yes, because they learn patterns from biased data.

17. Practice Tasks

- Build an `nn.Embedding` classifier for text labels.
- Compare cosine similarity of sentence embeddings.
- Visualize embeddings with PCA or t-SNE.
- Debug a retrieval system using mismatched embedding models.
- Fine-tune embeddings on domain-specific text.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Semantic FAQ Search	Retrieves closest FAQ answer	SentenceTransformers, FAISS	Company FAQ	Direct RAG relevance
Resume Matcher	Matches resumes to jobs	Python, embeddings	Kaggle resumes/jobs	Placement-friendly
Embedding Visualizer	Plots semantic clusters	PyTorch, sklearn	News/articles	Shows interpretability

19. Quick Revision

- Key idea: learned dense vectors capture useful similarity.
- Main formula: $x_i = E[\text{token_id_i}]$.
- When to use: whenever discrete inputs feed neural models.
- Important metrics: downstream score, recall@k, cosine similarity.
- Common traps: mismatched embedding models, bad pooling, bias.
- Interview one-liner: embeddings turn symbols into geometry.

20. Final Cheat Sheet

Item	Summary
Definition	Dense vector representation
Input/output	IDs/text -> vectors
Main steps	lookup, train, compare/use
Hyperparameters	dimension, vocabulary, normalization
Metrics	accuracy, recall@k, MRR, cosine
Pros	captures similarity, trainable
Cons	memory cost, bias, weak exactness
Best use cases	LLM inputs, retrieval, recommendation

Positional Encoding

1. Overview

Positional encoding adds order information to token embeddings. Self-attention by itself treats inputs like a set: without positions, "dog bites man" and "man bites dog" can look structurally similar. Positional encoding is used in Transformers for NLP, vision, audio, time series, and code.

2. Intuition

If token embeddings describe "what" each token is, positional encodings describe "where" each token is. It is like giving every word a seat number in a sentence.

3. Prerequisites

- Token embeddings
- Sine and cosine functions
- Matrix addition
- Self-attention
- Sequence modeling

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Absolute position	Position index from start	Encodes exact location	token 5	Simple but length-limited
Relative position	Distance between tokens	Better for relationships	previous token	Useful for long context
Learned position	Trainable vectors	Flexible	BERT positions	May not extrapolate
Sinusoidal position	Fixed functions	No extra learned table	original Transformer	Can extrapolate better
Rotary position	Rotates Q/K vectors	Strong for LLMs	RoPE	Modern decoder LLMs

5. Algorithm / Working Process

1. Tokenize input.
2. Convert token IDs to embeddings.
3. Create position vectors for each sequence index.
4. Combine token and position information, often by addition.
5. Feed combined vectors into attention layers.

For RoPE, positions are applied inside attention by rotating query and key dimensions.

6. Mathematical Foundation

Original sinusoidal encoding:

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{(2i/d_{\text{model}})}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{(2i/d_{\text{model}})}) \end{aligned}$$

Input to Transformer:

$$z_i = \text{token_embedding}_i + \text{positional_encoding}_i$$

For relative positions, attention score may include a bias:

$$\text{score}(i, j) = q_i k_j^T / \sqrt{d_k} + b_{(i-j)}$$

7. Practical Implementation

```
import math
import torch

def sinusoidal_positional_encoding(max_len: int, d_model: int) -> torch.Tensor:
    positions = torch.arange(max_len).unsqueeze(1)
    div_terms = torch.exp(
        torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
    )
    pe = torch.zeros(max_len, d_model)
    pe[:, 0::2] = torch.sin(positions * div_terms)
    pe[:, 1::2] = torch.cos(positions * div_terms)
    return pe

tokens = torch.randn(2, 10, 64) # batch, seq_len, d_model
pe = sinusoidal_positional_encoding(10, 64)
tokens_with_pos = tokens + pe.unsqueeze(0)
print(tokens_with_pos.shape)
```

8. Code Explanation

The function creates one position vector per sequence index. Even dimensions use sine and odd dimensions use cosine. `unsqueeze(0)` broadcasts the positional table across the batch.

9. Training / Evaluation

Learned encodings are trained with the model. Evaluate whether the model handles longer sequences, reordered tokens, and tasks requiring syntax or chronology. For long-context models, test retrieval at different

positions because models may show lost-in-the-middle behavior.

10. Complexity and Cost

Absolute learned embeddings add $\text{max_position} * \text{d_model}$ parameters. Sinusoidal encodings add no trainable parameters. Relative and rotary approaches add small compute overhead but can improve long-range modeling.

11. Common Use Cases

- Word order in NLP
- Patch order in Vision Transformers
- Time order in forecasting
- Code sequence order
- Document layout and long-context modeling

12. Common Mistakes

- Thinking attention automatically understands order
- Using learned positions beyond trained length without adaptation
- Forgetting padding position handling
- Confusing token embeddings with position embeddings
- Ignoring long-context position extrapolation issues

13. Edge Cases / Limitations

Absolute learned encodings may fail beyond the maximum trained position. Fixed encodings may be less flexible. Long sequences can still be hard because attention cost grows quadratically.

14. Variations

Variation	What changes	When to use	Placement value
Sinusoidal	Fixed sine/cosine	Teaching/original Transformer	Important
Learned absolute	Trainable table	BERT-style models	Important
Relative bias	Adds distance bias	Translation/long text	Important
RoPE	Rotates Q/K by position	Modern LLMs	Very important
ALiBi	Linear attention bias	Length extrapolation	Advanced

15. Related Topics

Positional encoding connects to self-attention because attention needs order signals. It connects to context window because position methods affect extrapolation. It connects to encoder-only, decoder-only, and encoder-decoder architectures because all need some form of position information.

16. Interview Questions

1. Why do Transformers need positional encoding?
Attention alone is permutation-invariant.
2. What is sinusoidal positional encoding?
A fixed sine/cosine vector based on position and dimension.
3. Learned vs sinusoidal positions?
Learned is flexible; sinusoidal has no parameters and can extrapolate better.
4. What is RoPE?
Rotary positional embedding applied to query/key vectors.
5. Where are positions added in the original Transformer?
To token embeddings before attention layers.
6. What is relative position encoding?
Encoding based on distances between tokens.
7. Can learned positions handle longer sequences?
Not reliably without extension or fine-tuning.
8. Why does word order matter?
Different order can change meaning.
9. Does positional encoding solve all long-context problems?
No, attention cost and retrieval behavior remain issues.
10. What is ALiBi useful for?
Length extrapolation using linear attention biases.

17. Practice Tasks

- Implement sinusoidal positional encoding.
- Remove position encoding from a tiny Transformer and observe accuracy.
- Compare learned and sinusoidal positions on a sequence task.
- Inspect attention patterns for nearby vs distant tokens.
- Experiment with truncation and long inputs.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Position Encoding Lab	Compares position methods	PyTorch	Synthetic sequence tasks	Shows Transformer fundamentals
Long Document Probe	Tests performance by answer position	HF Transformers	QA docs	Relevant to RAG
ViT Patch Order Demo	Shows image patch position effects	PyTorch/torchvision	CIFAR-10	Bridges CV and Transformers

19. Quick Revision

- Key idea: add order information to tokens.
- Main formula: $z_i = e_i + PE_i$.
- When to use: all sequence Transformers.
- Important metrics: task score across sequence lengths.

- Common traps: assuming attention knows order.
- Interview one-liner: positional encoding gives Transformers a sense of sequence order.

20. Final Cheat Sheet

Item	Summary
Definition	Position signal for token vectors
Input/output	token embeddings -> position-aware embeddings
Main steps	compute position vectors, add/apply to embeddings
Hyperparameters	max length, position type
Metrics	generalization by length/position
Pros	enables order modeling
Cons	extrapolation limits
Best use cases	NLP, ViT, time series, code

Self-Attention

1. Overview

Self-attention lets each token look at other tokens in the same sequence and decide which ones are important. It is the central operation in Transformers. It powers LLMs, BERT classifiers, translation models, summarizers, code models, and multimodal models.

2. Intuition

In the sentence "The animal did not cross the road because it was tired", the model must decide what "it" refers to. Self-attention allows "it" to attend strongly to "animal" instead of unrelated words.

3. Prerequisites

- Vectors and matrix multiplication
- Softmax
- Embeddings
- Gradients and neural networks
- Sequence modeling

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Query Q	What token is looking for	Determines attention request	pronoun asks for referent	Q/K/V roles

Concept	Meaning	Why it matters	Example	Interview angle
Key K	What token offers	Used for matching	noun has matching key	Dot-product score
Value V	Information passed	Weighted and aggregated	noun representation	Output is value mixture
Attention score	Q-K similarity	Chooses important tokens	high score to subject	Scaling by $\sqrt{d_k}$
Mask	Blocks positions	Prevents padding/future access	causal mask	Decoder correctness

5. Algorithm / Working Process

Input: token representations **X**.

1. Project **X** into queries, keys, and values.
2. Compute similarity scores QK^T .
3. Scale scores by $\sqrt{d_k}$.
4. Add mask if needed.
5. Apply softmax to get attention weights.
6. Multiply weights by values.
7. Return context-aware token representations.

Output: each token becomes a weighted summary of relevant tokens.

6. Mathematical Foundation

Scaled dot-product attention:

$$\begin{aligned} Q &= XW_Q \\ K &= XW_K \\ V &= XW_V \end{aligned}$$

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

Softmax:

$$\text{softmax}(z_i) = \exp(z_i) / \sum_j \exp(z_j)$$

The scaling prevents large dot products from saturating softmax when d_k is large.

7. Practical Implementation

```
import math
import torch
import torch.nn.functional as F

def self_attention(x, mask=None):
    batch, seq_len, d_model = x.shape
    wq = torch.randn(d_model, d_model)
    wk = torch.randn(d_model, d_model)
    wv = torch.randn(d_model, d_model)

    q = x @ wq
    k = x @ wk
    v = x @ wv

    scores = q @ k.transpose(-2, -1) / math.sqrt(d_model)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float("-inf"))

    weights = F.softmax(scores, dim=-1)
    return weights @ v, weights

x = torch.randn(2, 5, 32)
out, attn = self_attention(x)
print(out.shape, attn.shape)
```

8. Code Explanation

The code creates query, key, and value projections. `q @ k.transpose(-2, -1)` computes token-to-token similarity. Softmax converts scores into weights. Multiplying by `v` creates the final context vector.

9. Training / Evaluation

Self-attention is trained end-to-end through the model loss. For classification, evaluate accuracy/F1. For language models, evaluate cross-entropy and perplexity. Attention maps can be inspected, but attention weights are not always faithful explanations.

10. Complexity and Cost

For sequence length `n` and hidden size `d`:

```
time complexity ≈ O(n^2 d)
memory complexity ≈ O(n^2)
```

The quadratic term comes from the `n × n` attention matrix.

11. Common Use Cases

- Long-range dependency modeling

- Machine translation
- Text classification
- Summarization
- Code generation
- Document question answering

12. Common Mistakes

- Forgetting the `sqrt(d_k)` scaling
- Applying softmax over the wrong dimension
- Incorrect mask shape
- Thinking attention weights always explain decisions
- Ignoring quadratic memory cost

13. Edge Cases / Limitations

Self-attention becomes expensive for long sequences. It may over-focus on spurious tokens. Without masks, decoder models can leak future information during training.

14. Variations

Variation	What changes	When to use	Placement value
Bidirectional attention	All tokens attend all tokens	BERT/encoders	Very important
Causal attention	Only previous tokens visible	GPT/decoders	Very important
Cross-attention	Queries attend external sequence	encoder-decoder/RAG-ish fusion	Important
Sparse attention	Limits token pairs	Long context	Advanced
FlashAttention	Memory-efficient exact attention	Faster LLM training/inference	Advanced but valuable

15. Related Topics

Self-attention is the base for multi-head attention. It depends on embeddings and positional encoding. It connects to causal language modeling through masks and to encoder-only models through bidirectional attention.

16. Interview Questions

1. What is self-attention?

A mechanism where each token computes a weighted summary of tokens in the same sequence.

2. What are Q, K, and V?

Query asks, key matches, value provides information.

3. Why scale by $\sqrt{d_k}$?
To keep dot-product magnitudes stable for softmax.
4. What is causal masking?
Blocking attention to future tokens.
5. What is attention complexity?
Roughly $O(n^2 d)$ time and $O(n^2)$ memory.
6. Why is self-attention better than RNNs for parallelism?
All token interactions can be computed with matrix operations.
7. Does self-attention understand order alone?
No, it needs positional information.
8. What is bidirectional attention?
Tokens can attend to both left and right context.
9. Why can attention be hard for long documents?
The attention matrix grows quadratically.
10. Are attention weights explanations?
They are useful diagnostics but not always faithful explanations.

17. Practice Tasks

- Implement scaled dot-product attention.
- Add a causal mask and verify future tokens receive zero probability.
- Visualize attention weights on a short sentence.
- Compare attention runtime as sequence length doubles.
- Debug a mask broadcasting error.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Tiny Attention Classifier	Uses attention for text classification	PyTorch	AG News	Shows fundamentals
Attention Visualizer	Displays token-token weights	Streamlit, PyTorch	Any text	Interview demo
Causal Mask Playground	Demonstrates autoregressive attention	PyTorch	Toy sequences	LLM concept clarity

19. Quick Revision

- Key idea: tokens dynamically weight other tokens.
- Main formula: $\text{softmax}(QK^T / \sqrt{d_k})V$.
- When to use: sequence modeling and Transformers.
- Important metrics: task loss, accuracy, perplexity.
- Common traps: mask errors and quadratic cost.
- Interview one-liner: self-attention lets each token build context from the whole visible sequence.

20. Final Cheat Sheet

Item	Summary
Definition	Token-to-token weighted interaction
Input/output	sequence vectors -> contextual vectors
Main steps	Q/K/V, scores, mask, softmax, weighted sum
Hyperparameters	hidden size, sequence length, mask type
Metrics	loss, perplexity, accuracy
Pros	parallel, captures long dependencies
Cons	quadratic cost
Best use cases	Transformers, LLMs, NLP, code

Multi-head Attention

1. Overview

Multi-head attention runs several attention operations in parallel, allowing different heads to learn different relationship patterns. One head may track syntax, another entity references, another local context. It is used in nearly every Transformer architecture.

2. Intuition

Instead of one person reading a sentence with one focus, imagine multiple reviewers: one checks grammar, one checks entities, one checks chronology. Their findings are combined.

3. Prerequisites

- Self-attention
- Matrix multiplication
- Tensor shapes
- Concatenation and linear projection
- Transformer architecture

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Head	Independent attention subspace	Learns diverse patterns	syntax head	Head dimension
Head dimension	<code>d_model / num_heads</code>	Controls per-head capacity	768/12=64	Divisibility

Concept	Meaning	Why it matters	Example	Interview angle
Concatenation	Join head outputs	Combines information	<code>[h1; h2; ...]</code>	Output shape
Output projection	Linear mix after concat	Integrates heads	<code>W_O</code>	Not just stacking
Head redundancy	Some heads may be similar	Pruning possible	unused heads	Interpretability

5. Algorithm / Working Process

1. Project input into Q, K, V.
2. Split Q, K, V into `h` heads.
3. Run scaled dot-product attention per head.
4. Concatenate head outputs.
5. Apply final output projection.

Output shape usually matches input shape: `(batch, seq_len, d_model)`.

6. Mathematical Foundation

```
head_i = Attention(XW_i^Q, XW_i^K, XW_i^V)
MultiHead(X) = Concat(head_1, ..., head_h) W^O
```

If `d_model = 768` and `h = 12`:

```
d_head = d_model / h = 64
```

7. Practical Implementation

```
import torch
import torch.nn as nn

mha = nn.MultiheadAttention(embed_dim=128, num_heads=8, batch_first=True)

x = torch.randn(4, 12, 128) # batch, seq_len, d_model
out, weights = mha(x, x, x, need_weights=True)

print(out.shape)      # [4, 12, 128]
print(weights.shape)  # [4, 12, 12] averaged over heads by default
```

8. Code Explanation

PyTorch's `MultiheadAttention` accepts query, key, and value. For self-attention all three are `x`. `batch_first=True` makes tensors easier to read as batch, sequence, hidden dimension.

9. Training / Evaluation

Multi-head attention is trained end-to-end. Evaluate with the model's task metric. You can inspect per-head attention patterns, ablate heads, or prune heads to study redundancy.

10. Complexity and Cost

Multi-head attention has similar asymptotic complexity to single attention with the same `d_model`:

```
time ≈ O(n^2 d_model)
memory ≈ O(h n^2)
```

More heads can improve expressiveness but increase attention-map memory.

11. Common Use Cases

- Transformer encoder and decoder blocks
- LLM generation
- Translation
- Vision Transformers
- Multimodal attention

12. Common Mistakes

- Choosing `num_heads` that does not divide `d_model`
- Confusing heads with layers
- Assuming every head is interpretable
- Forgetting output projection
- Using too many heads for a tiny model

13. Edge Cases / Limitations

Heads can become redundant. More heads do not guarantee better performance. Attention memory grows with number of heads and sequence length.

14. Variations

Variation	What changes	When to use	Placement value
MHA	Separate Q/K/V per head	Standard Transformer	Very important
MQA	Multiple query heads share K/V	Faster decoding	Important for LLM inference

Variation	What changes	When to use	Placement value
GQA	Groups of query heads share K/V	Balance quality/speed	Modern LLMs
Cross multi-head attention	Q from decoder, K/V from encoder	Translation/T5	Important
Flash MHA	Efficient kernel	Long/fast training	Advanced

15. Related Topics

Multi-head attention extends self-attention. It connects to decoder-only inference because KV cache size depends on heads. It connects to model cost because attention heads affect memory and speed.

16. Interview Questions

1. Why use multiple heads?
To let the model attend to different patterns in parallel.
2. How is head dimension computed?
 $d_{\text{head}} = d_{\text{model}} / \text{num_heads}$.
3. What happens after heads are computed?
They are concatenated and passed through output projection.
4. Does MHA change output dimension?
Usually no; output returns to d_{model} .
5. MHA vs self-attention?
MHA is several self-attention operations in parallel.
6. What is MQA?
Multi-query attention shares key/value heads to speed decoding.
7. What is GQA?
Grouped-query attention shares K/V within groups.
8. Why can heads be pruned?
Some heads may learn redundant behavior.
9. Why does MHA help language modeling?
Different heads capture syntax, semantics, and long-range dependencies.
10. What is a common shape bug?
Mixing batch-first and sequence-first tensor layouts.

17. Practice Tasks

- Use PyTorch MHA on toy data.
- Print tensor shapes inside a custom MHA implementation.
- Compare 1, 2, 4, and 8 heads on a small classifier.
- Visualize per-head attention.
- Implement causal MHA masking.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Head Visualizer	Shows attention by head	PyTorch, Streamlit	Sentences	Strong explainability demo
Tiny Transformer	Implements MHA from scratch	PyTorch	Tiny Shakespeare	Core LLM skill
MHA Benchmark	Measures speed/memory by heads	PyTorch	Synthetic sequences	Systems awareness

19. Quick Revision

- Key idea: several attention views in parallel.
- Main formula: $\text{Concat}(\text{head}_i)W^O$.
- When to use: standard Transformer blocks.
- Important metrics: loss, speed, memory.
- Common traps: shape errors, overinterpreting heads.
- Interview one-liner: multi-head attention gives the model multiple learned ways to look at the same sequence.

20. Final Cheat Sheet

Item	Summary
Definition	Parallel attention heads
Input/output	sequence vectors -> same-shaped contextual vectors
Main steps	project, split heads, attend, concat, project
Hyperparameters	number of heads, head dimension
Metrics	task score, latency, memory
Pros	richer relationships
Cons	memory and redundancy
Best use cases	all Transformer architectures

Feed-forward Block

1. Overview

The feed-forward block, also called MLP block or FFN, is the position-wise neural network inside each Transformer layer. Attention mixes information across tokens; the feed-forward block transforms each token independently to increase model capacity.

2. Intuition

Attention lets tokens talk to each other. The feed-forward block lets each token "think" after receiving context. It applies the same small neural network to every token position.

3. Prerequisites

- Linear layers
- Activation functions such as ReLU, GELU, SiLU
- Matrix multiplication
- Backpropagation
- Transformer block structure

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Position-wise	Same network per token	Efficient and parallel	token 1 and 2 share FFN	No token mixing here
Expansion	Hidden size grows	Adds capacity	d_model -> 4d	FFN often has many params
Activation	Nonlinearity	Enables complex functions	GELU	ReLU vs GELU
Dropout	Regularization	Reduces overfitting	after activation	Training only
Gating	Multiplicative path	Modern LLM performance	SwiGLU	Advanced LLMs

5. Algorithm / Working Process

1. Receive contextual token vectors from attention.
2. Apply first linear projection to larger hidden size.
3. Apply activation.
4. Optionally apply dropout or gating.
5. Project back to d_model.
6. Add residual connection and normalization depending on architecture.

6. Mathematical Foundation

Original Transformer FFN:

FFN(x) = max(0, xW_1 + b_1)W_2 + b_2

GELU activation:

GELU(x) ≈ 0.5x(1 + tanh(sqrt(2/pi)(x + 0.044715x^3)))

SwiGLU-style block:

$$\text{FFN}(x) = (\text{SiLU}(xW_g) * xW_u) W_d$$

7. Practical Implementation

```
import torch
import torch.nn as nn

class FeedForward(nn.Module):
    def __init__(self, d_model: int, expansion: int = 4, dropout: float = 0.1):
        super().__init__()
        hidden = d_model * expansion
        self.net = nn.Sequential(
            nn.Linear(d_model, hidden),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(hidden, d_model),
        )

    def forward(self, x):
        return self.net(x)

ffn = FeedForward(d_model=128)
x = torch.randn(2, 10, 128)
print(ffn(x).shape)
```

8. Code Explanation

The first linear layer expands the representation. GELU adds nonlinearity. Dropout regularizes during training. The second linear layer returns to the original hidden size so the block can be used with residual connections.

9. Training / Evaluation

The FFN is trained with the full Transformer. It often contains a large fraction of model parameters, so changing FFN width strongly affects capacity. Watch validation loss, overfitting, and GPU memory.

10. Complexity and Cost

For sequence length n , model dimension d , and expansion factor r :

$$\begin{aligned} \text{FFN parameters} &\approx 2 * d * (r * d) \\ \text{time} &\approx O(n * r * d^2) \end{aligned}$$

FFN cost can dominate for short sequences; attention dominates more as sequence length grows.

11. Common Use Cases

- Transformer encoder/decoder layers
- LLM hidden transformations
- Vision Transformer MLP blocks
- Adapter and LoRA target modules

12. Common Mistakes

- Thinking FFN mixes tokens
- Forgetting activation function
- Making hidden size too small
- Ignoring FFN parameter cost
- Applying dropout during evaluation accidentally

13. Edge Cases / Limitations

Very wide FFNs increase memory and overfitting risk. Too narrow FFNs bottleneck model capacity. For deployment, FFN matrix multiplications can be a major latency cost.

14. Variations

Variation	What changes	When to use	Placement value
ReLU FFN	Original simple nonlinearity	Teaching	Basic
GELU FFN	Smooth activation	BERT/GPT-style	Important
SwiGLU/GeGLU	Gated activation	Modern LLMs	Advanced
Mixture of Experts	Routes tokens to experts	Huge sparse models	Research/high-scale
Adapter FFN	Small inserted MLP	Parameter-efficient tuning	Projects

15. Related Topics

The FFN connects to residual connections and layer normalization because it is wrapped by both. It complements self-attention: attention mixes tokens; FFN transforms each token.

16. Interview Questions

1. What is the FFN block?
A position-wise MLP inside each Transformer layer.
2. Does FFN mix information across tokens?
No, attention does token mixing.
3. Why expand hidden dimension?
To increase representational capacity.
4. Common expansion factor?
Often around 4 in classic Transformers.
5. Why use GELU?
It is a smooth nonlinearity that works well in Transformer models.

6. What is SwiGLU?

A gated FFN variant used in modern LLMs.

7. Why project back to `d_model`?

To preserve shape for residual connections.

8. Is FFN expensive?

Yes, it can hold many parameters.

9. What happens if activation is removed?

Stacked linear layers collapse into one linear transformation.

10. FFN vs attention?

FFN transforms token features; attention exchanges information across tokens.

17. Practice Tasks

- Implement FFN with ReLU, GELU, and SwiGLU.
- Count FFN parameters in a Transformer layer.
- Train a tiny Transformer with different expansion ratios.
- Benchmark FFN latency on CPU vs GPU.
- Add dropout and compare validation loss.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
FFN Ablation Lab	Compares activations/widths	PyTorch	Text classification	Shows research mindset
Tiny GPT MLP Swap	Replaces GELU with SwiGLU	PyTorch	Tiny Shakespeare	LLM architecture skill
Parameter Counter	Reports attention vs FFN params	Python	Model configs	Useful engineering tool

19. Quick Revision

- Key idea: per-token MLP after attention.
- Main formula: $\text{FFN}(x) = \text{activation}(xW_1 + b_1)W_2 + b_2$.
- When to use: every Transformer block.
- Important metrics: validation loss, latency, parameter count.
- Common traps: assuming it mixes tokens.
- Interview one-liner: attention communicates; FFN computes.

20. Final Cheat Sheet

Item	Summary
Definition	Position-wise MLP in Transformer
Input/output	contextual vectors -> transformed vectors

Item	Summary
Main steps	expand, activate, dropout/gate, project
Hyperparameters	expansion ratio, activation, dropout
Metrics	loss, speed, memory
Pros	high capacity
Cons	parameter-heavy
Best use cases	Transformer layers

Residual Connection

1. Overview

A residual connection adds a block's input to its output. In Transformers, attention and FFN sublayers are usually wrapped with residual paths. Residuals improve gradient flow, stabilize deep networks, and make it easier to train large models.

2. Intuition

Instead of forcing a layer to learn a complete transformation, residuals let it learn a correction. If no change is needed, the block can learn near-zero output and pass the input through.

3. Prerequisites

- Neural networks and backpropagation
- Gradients and vanishing gradients
- Tensor shape compatibility
- Transformer blocks

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Skip path	Direct input route	Helps gradients	$x + f(x)$	Deep training
Shape match	Same dimensions needed	Addition requires it	both <code>d_model</code>	Projection if dimensions differ
Identity mapping	Pass-through behavior	Easier optimization	output near zero	ResNet idea
Pre-norm	Norm before sublayer	Stable large training	$x + f(\text{LN}(x))$	Modern LLMs
Post-norm	Norm after residual	Original Transformer	$\text{LN}(x + f(x))$	Can be less stable deep

5. Algorithm / Working Process

For each sublayer:

1. Take input x .
2. Compute sublayer output $f(x)$ or $f(\text{LN}(x))$.
3. Add original input: $x + f(x)$.
4. Optionally apply layer normalization depending on architecture.

6. Mathematical Foundation

Residual block:

$$y = x + F(x)$$

Gradient:

$$dL/dx = dL/dy * (1 + dF/dx)$$

The direct 1 term helps gradients flow backward even if dF/dx is small.

7. Practical Implementation

```
import torch
import torch.nn as nn

class ResidualFFNBlock(nn.Module):
    def __init__(self, d_model: int):
        super().__init__()
        self.norm = nn.LayerNorm(d_model)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, 4 * d_model),
            nn.GELU(),
            nn.Linear(4 * d_model, d_model),
        )

    def forward(self, x):
        return x + self.ffn(self.norm(x))  # pre-norm residual

x = torch.randn(2, 8, 64)
block = ResidualFFNBlock(64)
print(block(x).shape)
```

8. Code Explanation

The block normalizes input first, passes it through an FFN, then adds the original x . The output shape remains unchanged, which allows stacking many layers.

9. Training / Evaluation

Residuals are not separately evaluated; they improve trainability. Watch gradient norms, convergence speed, training stability, and ability to train deeper models.

10. Complexity and Cost

Residual addition adds negligible compute and memory compared with attention and FFN. It requires storing activations for backpropagation like other layers.

11. Common Use Cases

- Transformers
- ResNets
- Deep CNNs
- Diffusion models
- Very deep MLPs

12. Common Mistakes

- Adding tensors with mismatched shapes
- Removing residuals from deep models
- Confusing residuals with normalization
- Forgetting dropout placement in residual branches
- Assuming residuals solve all instability problems

13. Edge Cases / Limitations

Residuals require compatible dimensions. Very deep networks may still need careful initialization, normalization, learning-rate scheduling, and gradient clipping.

14. Variations

Variation	What changes	When to use	Placement value
Basic residual	$x + F(x)$	Most deep nets	Important
Projected residual	projection aligns shapes	dimension changes	Important
Pre-norm residual	norm before sublayer	modern LLMs	Very important
Post-norm residual	norm after addition	original Transformer	Important
Gated residual	learned gate controls skip	advanced stability	Research

15. Related Topics

Residual connections connect closely to layer normalization. They wrap self-attention and FFN blocks. They also relate to ResNet, gradient flow, and deep model optimization.

16. Interview Questions

1. What is a residual connection?

A skip connection that adds input to a block's output.
2. Why are residuals useful?

They improve gradient flow and deep model training.
3. Formula for residual block?

$y = x + F(x)$.
4. What shape constraint exists?

x and $F(x)$ must have the same shape.
5. Pre-norm vs post-norm?

Pre-norm normalizes before the sublayer; post-norm after residual addition.
6. Why can a residual block learn identity?

If $F(x)$ becomes near zero, output is approximately x .
7. Are residuals only for Transformers?

No, they are common in ResNets and many deep models.
8. Do residuals add many parameters?

No, basic residual addition adds none.
9. Can residuals cause issues?

Scale can grow if not paired with normalization.
10. Why are residuals important for LLMs?

LLMs stack many layers and need stable optimization.

17. Practice Tasks

- Train a deep MLP with and without residuals.
- Implement pre-norm and post-norm Transformer blocks.
- Plot gradient norms across layers.
- Debug shape mismatch in a residual path.
- Add a projection residual when dimensions differ.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Residual Training Demo	Shows convergence improvement	PyTorch	MNIST	Clear fundamentals
Norm Placement Study	Compares pre/post norm	PyTorch	Toy LM	LLM training relevance
Deep MLP Stabilizer	Uses residuals for tabular modeling	PyTorch	UCI tabular	Practical architecture skill

19. Quick Revision

- Key idea: learn correction, not full transformation.
- Main formula: $y = x + F(x)$.
- When to use: deep networks.
- Important metrics: stability, loss, gradient norm.
- Common traps: shape mismatch.
- Interview one-liner: residuals give gradients and information a direct path through deep models.

20. Final Cheat Sheet

Item	Summary
Definition	Skip-add connection
Input/output	tensor -> same-shaped tensor
Main steps	compute sublayer, add input
Hyperparameters	usually none
Metrics	training stability, convergence
Pros	stable deep learning
Cons	shape constraints, scale issues
Best use cases	Transformers and deep nets

Layer Normalization

1. Overview

Layer normalization normalizes features within each example and token representation. In Transformers, it stabilizes training by keeping activations in a controlled range. It is preferred over batch normalization for sequence models because it does not depend on batch statistics.

2. Intuition

LayerNorm adjusts each token's feature vector so its values have stable mean and variance. It is like keeping every token representation on a consistent scale before or after a transformation.

3. Prerequisites

- Mean and variance
- Neural network activations
- BatchNorm basics
- Transformer block structure
- Trainable scale and shift parameters

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Feature-wise norm	Normalize hidden dimension	Stable per token	vector length 768	Unlike BatchNorm
Gamma/beta	Learnable scale/shift	Restores flexibility	<code>gamma, beta</code>	Trainable params
Epsilon	Numerical stability	Avoids divide by zero	<code>1e-5</code>	Implementation detail
Pre-norm	LN before sublayer	Stable deep LLMs	<code>x + f(LN(x))</code>	Modern default
Post-norm	LN after residual	Original Transformer	<code>LN(x + f(x))</code>	Training stability

5. Algorithm / Working Process

For each token vector:

1. Compute mean across hidden features.
2. Compute variance across hidden features.
3. Normalize each feature.
4. Apply learned scale `gamma`.
5. Apply learned shift `beta`.

6. Mathematical Foundation

For feature vector `x`:

```
mu = (1/d) sum_i x_i
sigma^2 = (1/d) sum_i (x_i - mu)^2
x_hat_i = (x_i - mu) / sqrt(sigma^2 + epsilon)
y_i = gamma_i x_hat_i + beta_i
```

LayerNorm does not average across batch examples.

7. Practical Implementation

```
import torch
import torch.nn as nn

layer_norm = nn.LayerNorm(normalized_shape=128)

x = torch.randn(4, 10, 128) # batch, seq_len, hidden
y = layer_norm(x)

print(y.mean(dim=-1)) # close to 0
print(y.var(dim=-1, unbiased=False)) # close to 1
```

8. Code Explanation

`nn.LayerNorm(128)` normalizes the last dimension. For Transformer tensors shaped `[batch, seq_len, hidden]`, each token vector is normalized independently.

9. Training / Evaluation

LayerNorm improves stability, especially for deep Transformers. Evaluate using convergence, validation loss, gradient stability, and ability to train at larger learning rates.

10. Complexity and Cost

LayerNorm cost is $O(\text{batch} * \text{sequence_length} * \text{hidden_dim})$, small compared with attention and FFN. It adds $2 * \text{hidden_dim}$ parameters for scale and shift.

11. Common Use Cases

- Transformer layers
- LLMs
- RNN replacements
- Diffusion models
- Small-batch or variable-length sequence training

12. Common Mistakes

- Confusing LayerNorm with BatchNorm
- Normalizing over the wrong dimension
- Forgetting epsilon
- Assuming it uses batch statistics
- Ignoring pre-norm vs post-norm differences

13. Edge Cases / Limitations

LayerNorm adds compute overhead and may not be optimal for all architectures. Poor norm placement can destabilize very deep models. Some modern variants remove bias or use RMSNorm.

14. Variations

Variation	What changes	When to use	Placement value
LayerNorm	mean and variance normalization	Standard Transformers	Very important
RMSNorm	normalizes RMS, no mean subtraction	Modern LLMs	Important
BatchNorm	batch statistics	CNNs	Compare in interviews
GroupNorm	groups channels	CV/small batch	Useful comparison
ScaleNorm	vector norm scaling	Transformer variant	Advanced

15. Related Topics

LayerNorm connects to residual connections, optimization, gradient stability, BatchNorm, RMSNorm, and Transformer block design.

16. Interview Questions

1. What is LayerNorm?
Normalization across features within each example/token.
2. How is it different from BatchNorm?
BatchNorm uses batch statistics; LayerNorm does not.
3. Formula for LayerNorm?
Subtract feature mean, divide by feature standard deviation, then scale and shift.
4. Why is LayerNorm common in Transformers?
It works well for variable-length sequences and small batches.
5. What are gamma and beta?
Learnable scale and shift.
6. What is epsilon for?
Numerical stability.
7. What is pre-norm?
Applying LayerNorm before attention/FFN.
8. Why do modern LLMs use pre-norm or RMSNorm?
Better stability for deep models.
9. Does LayerNorm behave differently in train and eval?
Not like BatchNorm; it uses current example statistics in both.
10. What dimension is normalized in `[B, T, D]`?
Usually `D`, the hidden dimension.

17. Practice Tasks

- Implement LayerNorm from scratch.
- Compare BatchNorm and LayerNorm on sequence data.
- Train tiny Transformer with pre-norm and post-norm.
- Replace LayerNorm with RMSNorm.
- Debug wrong normalized shape.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Norm From Scratch	Implements norms manually	PyTorch	Synthetic tensors	Strong math clarity
Transformer Stability Lab	Compares norm placement	PyTorch	Tiny LM	LLM training insight
RMSNorm Mini LLM	Builds modern norm variant	PyTorch	Tiny Shakespeare	Modern architecture detail

19. Quick Revision

- Key idea: normalize each token's hidden features.
- Main formula: $(x - \text{mean}) / \sqrt{\text{var} + \text{eps}}$.
- When to use: Transformers and sequence models.
- Important metrics: stable loss, gradient behavior.
- Common traps: wrong dimension, BatchNorm confusion.
- Interview one-liner: LayerNorm stabilizes each token representation without relying on batch statistics.

20. Final Cheat Sheet

Item	Summary
Definition	Feature normalization per example/token
Input/output	hidden vector -> normalized vector
Main steps	mean, variance, normalize, scale, shift
Hyperparameters	epsilon, normalized shape
Metrics	convergence, validation loss
Pros	stable, batch-size independent
Cons	extra compute, design choices
Best use cases	Transformers/LLMs

Encoder-only Models

1. Overview

Encoder-only models use Transformer encoder blocks with bidirectional self-attention. They are best for understanding tasks where the entire input is available: classification, named entity recognition, semantic similarity, reranking, and extractive question answering. BERT is the most famous example.

2. Intuition

An encoder-only model reads the full sentence before making a decision. It is like a student reading the whole paragraph before answering a comprehension question.

3. Prerequisites

- Transformer encoder block
- Bidirectional self-attention
- Tokenization and embeddings
- Classification heads
- Masked language modeling

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Bidirectional attention	sees left and right context	strong understanding	BERT	Not for autoregressive generation
[CLS] token	pooled sequence representation	classification	sentiment label	Pooling strategies
MLM	predict masked tokens	pretraining objective	[MASK]	BERT training
Token classification	label each token	NER/POS	BIO tags	Subword label alignment
Reranking	score query-document pair	search quality	cross-encoder	Accuracy vs latency

5. Algorithm / Working Process

1. Tokenize full input.
2. Add special tokens such as [CLS] and [SEP].
3. Convert to embeddings with positions.
4. Pass through stacked encoder blocks.
5. Use [CLS], pooled output, or token outputs for task head.
6. Train with supervised loss or MLM.

Inference output depends on task: class label, token labels, span, or similarity score.

6. Mathematical Foundation

Bidirectional attention uses no causal mask:

$$H = \text{Encoder}(X)$$

Classification:

$$\begin{aligned} \text{logits} &= W h_{[\text{CLS}]} + b \\ P(y|x) &= \text{softmax}(\text{logits}) \\ L &= -\log P(y_{\text{true}}|x) \end{aligned}$$

Masked language modeling:

$$L_{\text{MLM}} = - \sum_{i \text{ in masked_positions}} \log P(x_i \mid x_{\text{visible}})$$

7. Practical Implementation

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name,
num_labels=2)

text = "The interview preparation guide is very useful."
inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True)

with torch.no_grad():
    outputs = model(**inputs)
    probs = torch.softmax(outputs.logits, dim=-1)

print(probs)
```

8. Code Explanation

The tokenizer prepares input IDs and masks. The encoder model reads the entire sentence bidirectionally. The classification head converts the pooled representation into class logits.

9. Training / Evaluation

Use train/validation/test splits. Common metrics include accuracy, F1, precision, recall, ROC-AUC for classification; token-level F1 for NER; exact match/F1 for extractive QA. Avoid leakage across similar documents.

10. Complexity and Cost

Encoder attention cost is $O(n^2 d)$ because all tokens attend to all tokens. Encoders are cheaper than huge decoder LLMs for understanding tasks and often faster for classification.

11. Common Use Cases

- Sentiment analysis
- Spam detection
- NER
- Extractive QA
- Semantic reranking
- Intent classification

12. Common Mistakes

- Using encoder-only models for free-form generation
- Forgetting token-label alignment in NER
- Overusing [CLS] when mean pooling works better for embeddings
- Fine-tuning with too high learning rate

- Evaluating on leaked duplicate text

13. Edge Cases / Limitations

Encoder-only models do not naturally generate text autoregressively. They are limited by max sequence length and can be slower than bi-encoder retrieval for large-scale search.

14. Variations

Variation	What changes	When to use	Placement value
BERT	classic encoder	understanding tasks	Very important
RoBERTa	improved BERT training	strong classification	Important
DistilBERT	compressed BERT	low latency	Projects
DeBERTa	disentangled attention	high accuracy	Advanced
Sentence-BERT	sentence embeddings	semantic search	Very important

15. Related Topics

Encoder-only models contrast with decoder-only models. They connect to fine-tuning, embeddings, masked language modeling, and reranking in RAG.

16. Interview Questions

1. What is an encoder-only model?
A Transformer stack using bidirectional self-attention for understanding tasks.
2. Example?
BERT, RoBERTa, DistilBERT.
3. Why is BERT bidirectional?
Tokens attend to both left and right context.
4. Can BERT generate text like GPT?
Not naturally autoregressively.
5. What is MLM?
Predicting masked tokens from surrounding context.
6. What is **[CLS]** used for?
Sequence-level classification representation.
7. Best tasks for encoders?
Classification, NER, extractive QA, reranking.
8. Encoder vs decoder attention mask?
Encoder uses full attention; decoder uses causal attention.
9. Why use DistilBERT?
Faster and smaller with some accuracy trade-off.
10. What metric for NER?
Entity-level F1 is common.

17. Practice Tasks

- Fine-tune DistilBERT for sentiment classification.
- Build a BERT NER model and align subword labels.
- Compare [CLS] pooling and mean pooling.
- Use a cross-encoder for search reranking.
- Debug overfitting on a small text dataset.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Interview Intent Classifier	Classifies candidate queries	HF, PyTorch	Custom intents	Practical NLP app
Resume NER	Extracts skills and education	BERT, spaCy	Resume dataset	Placement-focused
Search Reranker	Improves BM25 results	Cross-encoder	MS MARCO sample	RAG relevance

19. Quick Revision

- Key idea: bidirectional Transformer for understanding.
- Main formula: $\text{softmax}(W \cdot h_{[\text{CLS}]} + b)$.
- When to use: classification/extraction/reranking.
- Important metrics: F1, accuracy, EM.
- Common traps: using it for generation.
- Interview one-liner: encoder-only models read the whole input to produce strong representations.

20. Final Cheat Sheet

Item	Summary
Definition	Bidirectional Transformer encoder model
Input/output	text -> labels/spans/representations
Main steps	tokenize, encode, task head
Hyperparameters	max length, LR, batch size
Metrics	accuracy, F1, EM
Pros	strong understanding
Cons	not natural for generation
Best use cases	classification, NER, QA, reranking

Decoder-only Models

1. Overview

Decoder-only models use causal self-attention to predict the next token. GPT-style LLMs are decoder-only. They are used for chatbots, code generation, summarization, reasoning, agents, data extraction, and creative writing.

2. Intuition

A decoder-only model writes one token at a time. It can only look at what has already been written, then predicts the next token.

3. Prerequisites

- Causal self-attention
- Language modeling
- Tokenization
- Probability distributions
- Sampling methods

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Autoregressive	predicts next token sequentially	generation	$P(x_t$	$x_{<t})$
Causal mask	blocks future tokens	prevents leakage	triangular mask	Training correctness
KV cache	stores previous keys/values	speeds inference	chat generation	Memory vs latency
Logits	raw token scores	sampling input	vocab-sized vector	Temperature/top-p
Prompt continuation	output follows prompt	flexible tasks	few-shot prompt	In-context learning

5. Algorithm / Working Process

Training:

1. Tokenize text.
2. Feed sequence with causal mask.
3. Predict each next token in parallel during training.
4. Minimize cross-entropy loss.

Inference:

1. Encode prompt.
2. Predict next-token distribution.
3. Choose token using greedy, temperature, top-k, or top-p sampling.

4. Append token.
5. Repeat until stop token or length limit.

6. Mathematical Foundation

Autoregressive factorization:

$$P(x_1, \dots, x_T) = \prod_t P(x_t \mid x_1, \dots, x_{t-1})$$

Training loss:

$$L = - \sum_t \log P(x_t \mid x_{<t})$$

Causal attention mask:

$$\text{mask}[i,j] = 1 \text{ if } j \leq i \text{ else } 0$$

7. Practical Implementation

```
from transformers import AutoTokenizer, AutoModelForCausalLM

model_name = "distilgpt2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

prompt = "In a machine learning interview, self-attention is"
inputs = tokenizer(prompt, return_tensors="pt")

outputs = model.generate(
    **inputs,
    max_new_tokens=40,
    do_sample=True,
    temperature=0.8,
    top_p=0.9,
)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

8. Code Explanation

`AutoModelForCausalLM` loads a next-token prediction model. `generate` repeatedly predicts and appends tokens. `temperature` and `top_p` control randomness and diversity.

9. Training / Evaluation

Pretraining uses massive unlabeled text. Evaluate with cross-entropy, perplexity, benchmark accuracy, human preference, pass@k for code, and task-specific tests. Fine-tuning can improve instruction following or domain behavior.

10. Complexity and Cost

Training attention is $O(n^2 d)$. During generation, tokens are produced sequentially, so latency grows with output length. KV caching reduces repeated computation but consumes memory:

$$\text{KV memory} \approx \text{layers} * \text{sequence_length} * \text{hidden_size} * \text{batch}$$

11. Common Use Cases

- Chatbots
- Code completion
- Summarization
- Tool-using agents
- Synthetic data generation
- Reasoning assistants

12. Common Mistakes

- Forgetting generation is sequential
- Using too much temperature for factual tasks
- Exceeding context window
- Confusing pretraining with instruction tuning
- Expecting perfect factuality without retrieval or tools

13. Edge Cases / Limitations

Decoder-only models can hallucinate, repeat text, ignore instructions, struggle with long-context retrieval, and produce unsafe or biased content if not aligned.

14. Variations

Variation	What changes	When to use	Placement value
GPT-style dense LM	standard decoder	general generation	Very important
Code LM	trained on code	coding tasks	Important
MoE decoder	sparse experts	large-scale models	Advanced
Chat-tuned decoder	instruction/alignment tuned	assistants	Very important
Multimodal decoder	accepts image/audio tokens	VLMs	Research/project

15. Related Topics

Decoder-only models connect to prompting, context windows, sampling, pretraining, fine-tuning, instruction tuning, and RLHF/DPO.

16. Interview Questions

1. What is a decoder-only model?
A causal Transformer trained to predict next tokens.
2. Example?
GPT-style models.
3. Why use causal masks?
To prevent looking at future tokens.
4. What is autoregressive generation?
Generating one token at a time conditioned on previous tokens.
5. What is next-token prediction loss?
Cross-entropy over the true next token.
6. What is KV cache?
Cached keys/values from prior tokens for faster decoding.
7. Why is generation slower than encoding?
Output tokens are produced sequentially.
8. What controls creativity?
Temperature, top-k, top-p, penalties, and prompt.
9. Can decoder-only models do classification?
Yes, by prompting or adding heads, but encoders may be cheaper.
10. Why do they hallucinate?
They model likely text, not guaranteed truth.

17. Practice Tasks

- Generate text with different temperatures.
- Implement a causal mask.
- Fine-tune a tiny GPT model on domain text.
- Measure latency with different output lengths.
- Build a prompt-based classifier.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Interview Answer Generator	Drafts answers to ML questions	HF, Streamlit	Curated Q&A	Placement friendly
Tiny GPT Trainer	Trains small LM	PyTorch	Tiny Shakespeare	Core LLM knowledge
Code Comment Generator	Generates docstrings	Transformers	CodeSearchNet	AI engineer relevance

19. Quick Revision

- Key idea: predict next token using past context.
- Main formula: $P(x) = \text{product } P(x_t | x_{<t})$.
- When to use: generation/chat/code.
- Important metrics: perplexity, task score, latency.
- Common traps: future leakage, hallucination, context overflow.
- Interview one-liner: decoder-only LLMs are causal next-token predictors scaled to broad capabilities.

20. Final Cheat Sheet

Item	Summary
Definition	Autoregressive Transformer
Input/output	prompt -> generated continuation
Main steps	encode prompt, predict token, sample, repeat
Hyperparameters	max tokens, temperature, top-p, top-k
Metrics	perplexity, accuracy, human preference
Pros	flexible generation
Cons	hallucination, sequential latency
Best use cases	chat, code, generation

Encoder-decoder Models

1. Overview

Encoder-decoder models use an encoder to understand an input sequence and a decoder to generate an output sequence. They are also called sequence-to-sequence models. They are used in translation, summarization, speech recognition, text simplification, and structured generation. T5 and BART are common examples.

2. Intuition

The encoder reads the source text and builds a memory. The decoder writes the target text while looking at that memory.

3. Prerequisites

- Encoder-only and decoder-only models
- Cross-attention
- Teacher forcing
- Sequence-to-sequence losses
- Tokenization

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Encoder memory	source representations	decoder conditions on input	article vectors	Cross-attention
Decoder	generates output tokens	target sequence	summary	Causal mask
Cross-attention	decoder attends encoder	links source and target	translation alignment	Q from decoder, K/V from encoder
Teacher forcing	train with true previous tokens	stable training	shift-right labels	Exposure bias
Seq2seq objective	map input to output	conditional generation	$P(y$	$x)$

5. Algorithm / Working Process

Training:

1. Tokenize input and target.
2. Encoder processes full input bidirectionally.
3. Decoder receives shifted target tokens with causal mask.
4. Cross-attention lets decoder attend to encoder outputs.
5. Minimize target-token cross-entropy.

Inference:

1. Encode input once.
2. Start decoder with BOS token.
3. Generate output autoregressively.
4. Stop at EOS or length limit.

6. Mathematical Foundation

Conditional generation:

$$P(y|x) = \prod_t P(y_t \mid y_{<t}, \text{Encoder}(x))$$

Training loss:

$$L = - \sum_t \log P(y_t \mid y_{<t}, x)$$

Cross-attention:

```

Q = H_decoder W_Q
K = H_encoder W_K
V = H_encoder W_V
Attention = softmax(QK^T / sqrt(d_k))V

```

7. Practical Implementation

```

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

model_name = "t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

text = "summarize: Transformers use attention to model relationships between tokens."
inputs = tokenizer(text, return_tensors="pt")

summary_ids = model.generate(**inputs, max_new_tokens=30)
print(tokenizer.decode(summary_ids[0], skip_special_tokens=True))

```

8. Code Explanation

T5 treats tasks as text-to-text. The prefix "summarize:" tells the model the task. The encoder reads the input, and the decoder generates the summary.

9. Training / Evaluation

Use paired input-output data. Metrics depend on task: BLEU for translation, ROUGE for summarization, exact match for structured output, and human evaluation for quality. Watch exposure bias, hallucination, and train-test domain mismatch.

10. Complexity and Cost

Encoder cost is $O(n^2 d)$, decoder self-attention is $O(m^2 d)$, and cross-attention is $O(n m d)$ for input length n and output length m .

11. Common Use Cases

- Machine translation
- Summarization
- Text-to-SQL
- Question generation
- Speech-to-text
- Data-to-text generation

12. Common Mistakes

- Using decoder-only assumptions for seq2seq
- Forgetting to shift decoder labels
- Evaluating summarization only with loss
- Ignoring source truncation
- Using BLEU/ROUGE without qualitative checks

13. Edge Cases / Limitations

Encoder-decoder models require paired data for supervised training. They can hallucinate in summarization and may be slower than encoder-only models for classification.

14. Variations

Variation	What changes	When to use	Placement value
T5	text-to-text unified framework	many NLP tasks	Very important
BART	denoising seq2seq	summarization	Important
MarianMT	translation-focused	machine translation	Useful
Whisper-style	audio encoder/text decoder	speech recognition	Project/research
Vision encoder-text decoder	image captioning/OCR	multimodal tasks	Project value

15. Related Topics

Encoder-decoder models connect encoder-only understanding with decoder-only generation. They heavily use cross-attention and are central to sequence-to-sequence learning.

16. Interview Questions

1. What is an encoder-decoder model?
A model that encodes input and decodes output conditionally.
2. Example?
T5, BART, MarianMT.
3. What is cross-attention?
Decoder queries attend to encoder keys and values.
4. What is teacher forcing?
Training decoder with true previous target tokens.
5. Formula for seq2seq probability?
 $P(y|x) = \text{product } P(y_t | y_{<t}, x)$.
6. Best tasks?
Translation, summarization, text-to-text generation.
7. Encoder-only vs encoder-decoder?
Encoder-only predicts labels/spans; encoder-decoder generates target sequences.
8. Decoder-only vs encoder-decoder?
Decoder-only continues prompt; encoder-decoder explicitly conditions on encoded input.
9. What metric for summarization?
ROUGE plus human/qualitative evaluation.

10. What is exposure bias?

Train uses true previous tokens, inference uses model-generated tokens.

17. Practice Tasks

- Fine-tune T5-small for summarization.
- Build a translation demo.
- Compare T5 and GPT prompting for summarization.
- Debug decoder label shifting.
- Evaluate ROUGE and inspect bad summaries.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Meeting Summarizer	Converts transcript to summary	T5/BART	AMI/SAMSum	Very practical
Text Simplifier	Rewrites complex text	HF Transformers	WikiLarge	NLP product value
Text-to-SQL Demo	Converts question to SQL	T5	Spider subset	AI engineer signal

19. Quick Revision

- Key idea: encode source, decode target.
- Main formula: $P(y|x) = \text{product } P(y_t | y_{<t}, x)$.
- When to use: conditional generation.
- Important metrics: BLEU, ROUGE, EM.
- Common traps: label shifting, source truncation.
- Interview one-liner: encoder-decoder models are best when input and output are separate sequences.

20. Final Cheat Sheet

Item	Summary
Definition	Seq2seq Transformer
Input/output	source text -> target text
Main steps	encode, cross-attend, decode
Hyperparameters	max source/target length, beam size
Metrics	BLEU, ROUGE, EM
Pros	strong conditional generation
Cons	paired data and decoding cost
Best use cases	translation, summarization, text-to-text

Pretraining

1. Overview

Pretraining trains a model on large-scale general data before adapting it to specific tasks. In LLMs, pretraining usually means next-token prediction on massive text corpora. It gives models broad language, world knowledge, reasoning patterns, and coding ability.

2. Intuition

Pretraining is like reading a huge library before taking a specific exam. The model learns general patterns first, then can specialize through fine-tuning or prompting.

3. Prerequisites

- Neural network optimization
- Language modeling objectives
- Datasets and data cleaning
- Distributed training basics
- Cross-entropy loss

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Objective	training task	defines learned behavior	next-token prediction	Causal vs masked LM
Corpus	training data	quality drives model	web/books/code	Data curation
Scaling	larger data/model/compute	improves capability	Chinchilla ideas	Compute trade-off
Checkpointing	saving model state	recovery and fine-tuning	epoch checkpoints	Engineering
Data mixture	proportions of sources	affects skills	code/math/web	Bias and domain balance

5. Algorithm / Working Process

1. Collect and clean large corpus.
2. Train or choose tokenizer.
3. Convert text to token sequences.
4. Train model with self-supervised objective.
5. Monitor validation loss and stability.
6. Save checkpoints.
7. Evaluate on downstream and benchmark tasks.

6. Mathematical Foundation

Causal LM loss:

$$L = - (1/T) \sum_t \log P(x_t \mid x_{<t}; \theta)$$

Masked LM loss:

$$L = - \sum_{i \in M} \log P(x_i \mid x_{\text{not_masked}}; \theta)$$

Optimization:

$$\theta \leftarrow \theta - \eta * \text{grad}_{\theta} L$$

Perplexity:

$$\text{PPL} = \exp(\text{cross_entropy})$$

7. Practical Implementation

```
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    DataCollatorForLanguageModeling,
    Trainer,
    TrainingArguments,
)
from datasets import Dataset

texts = ["Transformers predict tokens.", "Pretraining learns general patterns."]
dataset = Dataset.from_dict({"text": texts})

tokenizer = AutoTokenizer.from_pretrained("distilgpt2")
tokenizer.pad_token = tokenizer.eos_token

def tokenize(batch):
    return tokenizer(batch["text"], truncation=True, padding="max_length",
max_length=32)

tokenized = dataset.map(tokenize, batched=True, remove_columns=["text"])
model = AutoModelForCausalLM.from_pretrained("distilgpt2")

args = TrainingArguments(
```

```
    output_dir="./tmp-pretrain-demo",
    per_device_train_batch_size=2,
    max_steps=2,
    logging_steps=1,
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=tokenized,
    data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False),
)

trainer.train()
```

8. Code Explanation

This is a tiny demonstration, not real pretraining. The dataset is tokenized into fixed-length sequences.

`DataCollatorForLanguageModeling` creates labels for causal LM training. `Trainer` runs a few optimization steps.

9. Training / Evaluation

Real pretraining needs train/validation splits, deduplication, toxic data filtering, data mixture control, checkpointing, distributed training, loss monitoring, and benchmark evaluation. Metrics include validation loss, perplexity, downstream task accuracy, contamination checks, and safety evaluations.

10. Complexity and Cost

Pretraining is the most expensive stage. Cost depends on parameter count, token count, sequence length, hardware, and precision. Training compute roughly scales with:

```
compute ≈ O(parameters * training_tokens)
```

11. Common Use Cases

- Foundation models
- Domain LMs for legal, medical, finance, or code
- Embedding models
- Vision-language models
- Speech models

12. Common Mistakes

- Calling small fine-tuning "pretraining"
- Ignoring data quality
- Training with duplicate evaluation data
- Using too little compute for model size

- Forgetting tokenizer-domain mismatch

13. Edge Cases / Limitations

Pretraining is expensive and can learn bias, memorized private data, toxic language, and factual inaccuracies. It does not guarantee instruction following.

14. Variations

Variation	What changes	When to use	Placement value
Causal LM	next-token prediction	GPT-style LLMs	Very important
Masked LM	masked token prediction	BERT encoders	Very important
Denoising	reconstruct corrupted text	BART/T5	Important
Contrastive pretraining	align representations	embeddings/CLIP	Important
Continued pretraining	train base model on domain data	domain adaptation	Projects

15. Related Topics

Pretraining comes before fine-tuning and instruction tuning. It relates to tokenization, objective functions, scaling laws, data engineering, MLOps, and evaluation.

16. Interview Questions

1. What is pretraining?
Large-scale self-supervised training before task adaptation.
2. Why is it useful?
It learns general representations from abundant unlabeled data.
3. Causal vs masked LM?
Causal predicts next token; masked predicts hidden tokens using both sides.
4. What is perplexity?
Exponentiated cross-entropy; lower means better token prediction.
5. Is pretraining supervised?
Usually self-supervised.
6. Why does data quality matter?
Bad data teaches bad behavior and wastes compute.
7. What is continued pretraining?
Further pretraining on domain data.
8. Does pretraining make a chat model?
Not by itself; instruction tuning/alignment are needed.
9. What is data contamination?
Evaluation examples appearing in training data.
10. Why is pretraining expensive?
Huge parameter counts and token counts require massive compute.

17. Practice Tasks

- Run a tiny causal LM training demo.
- Compare validation loss on clean vs noisy data.
- Continue pretraining a small LM on domain text.
- Estimate tokens and compute for a corpus.
- Check for duplicated examples in train/test.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Domain LM Adapter	Continued pretraining on niche text	HF, PyTorch	arXiv/legal docs	Shows LLM adaptation
Corpus Cleaner	Deduplicates and filters text	Python, pandas	Common Crawl sample	Data engineering signal
Tiny LM Dashboard	Tracks loss/perplexity	PyTorch, Streamlit	Tiny Shakespeare	Training literacy

19. Quick Revision

- Key idea: learn general patterns before specialization.
- Main formula: $L = -\sum \log P(x_t | x_{<t})$.
- When to use: building/adapting foundation models.
- Important metrics: loss, perplexity, benchmarks.
- Common traps: data contamination and bad corpus quality.
- Interview one-liner: pretraining turns raw unlabeled data into reusable model capability.

20. Final Cheat Sheet

Item	Summary
Definition	Large-scale self-supervised training
Input/output	corpus -> pretrained model
Main steps	clean, tokenize, train, validate, checkpoint
Hyperparameters	LR, batch size, tokens, sequence length
Metrics	loss, perplexity, benchmark score
Pros	broad transfer
Cons	huge cost, bias, memorization
Best use cases	foundation/domain models

Fine-tuning

1. Overview

Fine-tuning adapts a pretrained model to a specific task or domain using additional training. It is used for classification, extraction, summarization, domain language, instruction following, coding style, and enterprise assistants.

2. Intuition

If pretraining is general education, fine-tuning is job training. The model already knows language; fine-tuning teaches it the target format or domain behavior.

3. Prerequisites

- Pretrained models
- Supervised learning
- Loss functions and optimization
- Train/validation/test split
- PyTorch/Hugging Face

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Full fine-tuning	update all weights	maximum flexibility	BERT classifier	Cost/overfitting
PEFT	update few parameters	cheaper adaptation	LoRA	Very important
Task head	output layer	adapts to labels	classification head	Head initialization
Catastrophic forgetting	loses old capability	overtraining risk	domain-only tuning	Data mixture
Validation	checks generalization	avoids overfit	held-out set	Leakage

5. Algorithm / Working Process

1. Select pretrained model and tokenizer.
2. Prepare task-specific dataset.
3. Add task head or format examples as text.
4. Train on labeled or instruction data.
5. Validate and tune hyperparameters.
6. Test once on held-out data.
7. Save model/adapters and deploy.

6. Mathematical Foundation

Classification fine-tuning:

```
h = Model(x)
logits = Wh + b
L = - sum_c y_c log softmax(logits)_c
```

Causal instruction fine-tuning:

$$L = - \sum_{\{\text{answer tokens}\}} \log P(y_t \mid \text{prompt}, y_{<t})$$

LoRA approximation:

```
W' = W + Delta W
Delta W = BA
rank(A,B) = r << d
```

7. Practical Implementation

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification,
TrainingArguments, Trainer
from datasets import Dataset

data = {
    "text": ["great product", "bad experience", "very useful", "not good"],
    "label": [1, 0, 1, 0],
}
dataset = Dataset.from_dict(data).train_test_split(test_size=0.5, seed=42)

tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

def tokenize(batch):
    return tokenizer(batch["text"], truncation=True, padding="max_length",
max_length=32)

tokenized = dataset.map(tokenize, batched=True)
model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-
uncased", num_labels=2)

args = TrainingArguments(
    output_dir="./tmp-finetune-demo",
    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,
    max_steps=2,
)

trainer = Trainer(
    model=model,
    args=args,
```

```
train_dataset=tokenized["train"],
eval_dataset=tokenized["test"],
)
trainer.train()
```

8. Code Explanation

The dataset is split before training. The tokenizer converts text into model inputs.

`AutoModelForSequenceClassification` adds a classification head. `Trainer` performs fine-tuning.

9. Training / Evaluation

Use stratified splits for classification. Metrics include accuracy, F1, precision, recall, ROC-AUC, BLEU/ROUGE for generation, and human evaluation for assistant behavior. Tune learning rate, batch size, epochs, weight decay, warmup, max length, and class imbalance handling.

10. Complexity and Cost

Full fine-tuning stores gradients and optimizer states for all parameters. PEFT methods such as LoRA reduce trainable parameters and GPU memory. Inference cost is usually similar to the base model unless adapters add overhead.

11. Common Use Cases

- Sentiment/intent classification
- Domain-specific extraction
- Customer-support assistants
- Code style adaptation
- Medical/legal NLP
- Summarization style control

12. Common Mistakes

- Fine-tuning before trying prompting/RAG
- Too small or low-quality dataset
- Data leakage from duplicates
- Learning rate too high
- Evaluating only on training examples
- Forgetting base model license/deployment constraints

13. Edge Cases / Limitations

Fine-tuning does not reliably add fresh factual knowledge unless trained carefully. It can overfit, forget general capability, or learn formatting shortcuts. For frequently changing knowledge, RAG may be better.

14. Variations

Variation	What changes	When to use	Placement value
Full fine-tuning	updates all weights	high data/compute	Important
Linear probing	train only head	quick baseline	Useful
LoRA	low-rank adapters	cheap LLM tuning	Very important
QLoRA	quantized LoRA	low VRAM	Very important
Instruction fine-tuning	prompt-response data	assistant behavior	Very important

15. Related Topics

Fine-tuning relates to pretraining, instruction tuning, prompting, RAG, LoRA/QLoRA, evaluation, and MLOps deployment.

16. Interview Questions

1. What is fine-tuning?
Additional training of a pretrained model for a task/domain.
2. Fine-tuning vs pretraining?
Pretraining learns general patterns; fine-tuning adapts to a target task.
3. What is LoRA?
A parameter-efficient method using low-rank weight updates.
4. When not to fine-tune?
When prompting or RAG solves the problem cheaper.
5. What is catastrophic forgetting?
Losing prior capability after narrow training.
6. Why use validation data?
To detect overfitting and tune hyperparameters.
7. What learning rate for Transformers?
Usually small, often around $1e-5$ to $5e-5$ for full fine-tuning.
8. What metric for imbalanced classification?
F1, precision/recall, PR-AUC.
9. Does fine-tuning update tokenizer?
Usually no, unless domain tokenization is a major issue.
10. Fine-tuning vs RAG?
Fine-tuning changes behavior; RAG injects external knowledge.

17. Practice Tasks

- Fine-tune BERT for sentiment.
- Fine-tune T5 for summarization.
- Compare full fine-tuning and LoRA.
- Debug overfitting with a tiny dataset.
- Build an evaluation set for instruction outputs.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Support Ticket Classifier	Routes tickets	BERT, HF	Customer tickets	Industry NLP
Domain Assistant Adapter	LoRA-tunes responses	PEFT, LLM	Internal Q&A	LLM engineering
Resume Skill Extractor	Fine-tunes NER	BERT	Resume dataset	Placement direct

19. Quick Revision

- Key idea: adapt pretrained model.
- Main formula: task loss over pretrained model outputs.
- When to use: stable task behavior/domain style.
- Important metrics: validation/test task metrics.
- Common traps: leakage, overfitting, fine-tuning when RAG is enough.
- Interview one-liner: fine-tuning specializes a pretrained model with task-specific examples.

20. Final Cheat Sheet

Item	Summary
Definition	Task/domain adaptation training
Input/output	pretrained model + data -> adapted model
Main steps	prepare data, train, validate, test
Hyperparameters	LR, batch size, epochs, rank
Metrics	F1, accuracy, ROUGE, human eval
Pros	strong task behavior
Cons	data/compute risk, forgetting
Best use cases	classification, extraction, style/instruction behavior

Prompting

1. Overview

Prompting is the practice of giving instructions, context, examples, and constraints to an LLM at inference time. It is useful because many LLM behaviors can be controlled without training. Prompting is used in chatbots, RAG, agents, summarization, extraction, coding assistants, and evaluation.

2. Intuition

Prompting is like writing the exam question clearly. A vague question gives vague answers; a structured prompt gives the model a better path.

3. Prerequisites

- Decoder-only LLM behavior
- Tokenization and context window
- Sampling parameters
- Task framing
- Evaluation mindset

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Instruction	what to do	directs behavior	"summarize in bullets"	Specificity
Context	information to use	grounds answer	retrieved docs	RAG
Few-shot examples	demonstrations	teach format	input-output pairs	In-context learning
Constraints	output rules	reliability	JSON schema	Parsing
System prompt	high-priority behavior	assistant policy/style	role message	Prompt hierarchy

5. Algorithm / Working Process

1. Define the task and desired output.
2. Provide relevant context.
3. Add examples if format is non-obvious.
4. Specify constraints such as length, schema, tone, or citations.
5. Run model with suitable decoding settings.
6. Validate output.
7. Iterate prompt based on failures.

6. Mathematical Foundation

Prompting changes the conditional distribution:

$$P(\text{output} \mid \text{prompt})$$

Few-shot prompting provides demonstrations:

```
prompt = instruction + examples + new_input
```

The model still predicts next tokens:

$$y_t \sim P(y_t \mid \text{prompt}, y_{<t})$$

7. Practical Implementation

```
from transformers import pipeline

generator = pipeline("text-generation", model="distilgpt2")

prompt = """
Task: Explain self-attention for an ML interview.
Constraints:
- Use simple language.
- Include the formula.
- End with one interview one-liner.
Answer:
"""

result = generator(prompt, max_new_tokens=80, do_sample=True, temperature=0.7)
print(result[0]["generated_text"])
```

8. Code Explanation

The prompt contains task, constraints, and an answer cue. `temperature=0.7` allows some variety. In production, use stronger models and output validation.

9. Training / Evaluation

Prompting does not train weights. Evaluate prompts with golden examples, unit-style checks, human review, exact-match tests for structured extraction, factuality checks, and regression tests across prompt versions.

10. Complexity and Cost

Prompting cost is proportional to input and output tokens. Long prompts increase latency and price. Few-shot examples improve behavior but consume context.

11. Common Use Cases

- Question answering
- Summarization
- Data extraction
- Code generation
- RAG answer synthesis
- Agent tool planning

12. Common Mistakes

- Vague instructions

- Too many conflicting rules
- No output schema for structured tasks
- Depending on prompt alone for factual accuracy
- Not testing prompts on failure cases
- Putting huge irrelevant context

13. Edge Cases / Limitations

Prompts can be ignored, misinterpreted, or attacked through prompt injection. Prompting cannot reliably teach large new knowledge or fix deep model limitations.

14. Variations

Variation	What changes	When to use	Placement value
Zero-shot	instruction only	simple tasks	Important
Few-shot	examples included	format/pattern tasks	Very important
Chain-of-thought style	asks for reasoning	reasoning tasks	Know carefully
Structured prompting	schema/output constraints	extraction/API use	Very important
RAG prompting	retrieved context included	factual QA	Very important

15. Related Topics

Prompting connects to decoder-only models, context windows, temperature/top-k/top-p sampling, instruction tuning, RAG, and evaluation.

16. Interview Questions

1. What is prompting?
Controlling model behavior through input instructions and context.
2. Zero-shot vs few-shot?
Zero-shot uses no examples; few-shot includes examples.
3. Why use examples?
They demonstrate task format and expected behavior.
4. What is prompt injection?
Malicious or accidental text that tries to override instructions.
5. Prompting vs fine-tuning?
Prompting changes input; fine-tuning changes weights.
6. Why specify output format?
To make responses easier to parse and evaluate.
7. Does a longer prompt always help?
No, irrelevant context can distract and cost more.
8. What is system prompt?
A higher-priority instruction setting role and behavior.
9. How evaluate prompts?
Test sets, human review, schema validation, regression checks.

10. Why is prompting important in RAG?

It tells the model how to use retrieved context.

17. Practice Tasks

- Create zero-shot and few-shot prompts for classification.
- Build a JSON extraction prompt and validate output.
- Test prompt robustness against irrelevant context.
- Compare concise and verbose prompts.
- Design a RAG answer prompt with citations.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Prompt Evaluation Harness	Tests prompt versions	Python, pytest	Custom examples	AI engineering skill
Resume Parser Prompt	Extracts structured fields	LLM API, JSON schema	Resumes	Practical automation
RAG Prompt Studio	Compares RAG templates	Streamlit, embeddings	PDFs	Interview-ready demo

19. Quick Revision

- Key idea: guide the model through input text.
- Main formula: $P(\text{output} | \text{prompt})$.
- When to use: fast behavior control without training.
- Important metrics: accuracy, schema validity, factuality.
- Common traps: vague prompts and no evaluation.
- Interview one-liner: prompting is inference-time programming for LLM behavior.

20. Final Cheat Sheet

Item	Summary
Definition	Input instructions/context for LLM
Input/output	prompt -> generated answer
Main steps	task, context, examples, constraints, validate
Hyperparameters	prompt length, examples, decoding params
Metrics	task accuracy, validity, human score
Pros	cheap, fast iteration
Cons	brittle, limited reliability
Best use cases	chat, extraction, summarization, RAG

Context Window

1. Overview

The context window is the maximum number of tokens an LLM can consider in one request, including prompt, retrieved context, conversation history, tool outputs, and generated response budget. It determines how much information can be processed at once.

2. Intuition

The context window is the model's working memory for a single interaction. If the useful information does not fit, the model cannot directly use it.

3. Prerequisites

- Tokenization
- Prompting
- Transformer attention cost
- RAG chunking
- Decoding/generation length

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Token budget	max allowed tokens	prevents overflow	8k/32k/128k	Input + output
Truncation	cutting tokens	can lose key info	old chat removed	Silent failure
Retrieval budget	context allocated to docs	affects RAG	top chunks	Chunk selection
Lost-in-the-middle	middle context ignored	long-context weakness	answer in middle	Evaluation
Compression	summarize context	saves tokens	memory summary	Information loss

5. Algorithm / Working Process

1. Count tokens for system prompt, user prompt, history, tools, and documents.
2. Reserve output tokens.
3. Keep highest-value context.
4. Truncate, summarize, or retrieve selectively if over budget.
5. Send final prompt to model.
6. Monitor whether output hits max token limit.

6. Mathematical Foundation

Token budget constraint:

```
system_tokens + history_tokens + retrieved_tokens + user_tokens +  
max_output_tokens <= context_window
```

Attention cost:

```
time/memory grows roughly with  $n^2$  for standard attention
```

7. Practical Implementation

```
from transformers import AutoTokenizer  
  
tokenizer = AutoTokenizer.from_pretrained("gpt2")  
  
context_window = 1024  
reserved_output = 150  
prompt = "Explain Transformers for interviews. " * 80  
  
tokens = tokenizer.encode(prompt)  
available_input = context_window - reserved_output  
  
if len(tokens) > available_input:  
    tokens = tokens[:available_input]  
  
safe_prompt = tokenizer.decode(tokens)  
print("input tokens:", len(tokens))  
print("available for output:", reserved_output)
```

8. Code Explanation

The code counts prompt tokens and reserves space for output. If input is too long, it truncates to fit. Production systems usually truncate smarter by keeping recent conversation and relevant retrieved chunks.

9. Training / Evaluation

For long-context systems, evaluate answer accuracy at different document positions, chunk counts, and history lengths. RAG systems should measure recall@k, answer faithfulness, latency, and cost.

10. Complexity and Cost

Longer context increases cost and latency. Standard attention has quadratic memory/time with sequence length, though optimized kernels and long-context architectures reduce practical cost.

11. Common Use Cases

- Chat history management
- RAG prompt construction
- Long-document QA
- Codebase assistants
- Agent scratchpads

12. Common Mistakes

- Counting words instead of tokens
- Forgetting output token reservation
- Stuffing irrelevant retrieved chunks
- Assuming bigger context means better answers
- Letting important instructions be truncated

13. Edge Cases / Limitations

Models may underuse long context, especially middle sections. Very long prompts are expensive. Confidential or irrelevant context can hurt quality and safety.

14. Variations

Variation	What changes	When to use	Placement value
Short context	cheaper, faster	simple tasks	Basic
Long context	more tokens	document/code tasks	Important
Sliding window	local attention window	streaming/long docs	Advanced
RAG context	retrieved relevant chunks	factual systems	Very important
Summarized memory	compressed history	long chats	AI engineering

15. Related Topics

Context window connects to tokenization, prompting, RAG, attention complexity, sampling length, and chat memory design.

16. Interview Questions

1. What is context window?
Maximum tokens a model can process in one request.
2. Does it include output?
Yes, input plus generated tokens must fit model limits.
3. Why count tokens not words?
Models operate on tokens.
4. What happens when context is too long?
It is rejected or truncated.
5. Why can long context still fail?
Models may ignore or underweight some positions.

6. What is lost-in-the-middle?
Reduced use of information placed in the middle of long context.
7. How does RAG manage context?
It retrieves only relevant chunks.
8. Why reserve output tokens?
To avoid generation being cut off.
9. Bigger context vs fine-tuning?
Bigger context supplies more input; fine-tuning changes behavior.
10. How reduce context cost?
Summarize, retrieve selectively, compress prompts.

17. Practice Tasks

- Count tokens for different prompts.
- Build a context budget calculator.
- Test answer accuracy with evidence at start/middle/end.
- Compare chunk sizes in RAG.
- Implement conversation history truncation.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Prompt Budgeter	Calculates token/cost budget	Python, HF tokenizer	User prompts	Useful tool
Long Context QA Test	Evaluates answer position effects	LLM API, Streamlit	Long docs	RAG evaluation
Chat Memory Manager	Summarizes old turns	Python, LLM	Conversations	Agent engineering

19. Quick Revision

- Key idea: model's token-limited working memory.
- Main formula: total input + output $\leq$ window.
- When to use: every LLM call.
- Important metrics: token count, latency, answer accuracy.
- Common traps: no output reserve, irrelevant context.
- Interview one-liner: context window is the hard token budget for what an LLM can see and say.

20. Final Cheat Sheet

Item	Summary
Definition	Max tokens per request
Input/output	prompt/history/docs + output
Main steps	count, reserve, select, truncate/compress

Item	Summary
Hyperparameters	max input, max output, chunk count
Metrics	token count, cost, recall, accuracy
Pros	more information available
Cons	cost, latency, attention limits
Best use cases	RAG, long docs, chat memory

Temperature

1. Overview

Temperature is a decoding parameter that controls randomness in token sampling. Lower temperature makes output more deterministic; higher temperature makes output more diverse. It is used in LLM generation for chat, writing, brainstorming, code, and data generation.

2. Intuition

Temperature controls how adventurous the model is. At low temperature it picks safer words; at high temperature it explores less likely words.

3. Prerequisites

- Logits and softmax
- Probability distributions
- Decoder-only generation
- Sampling
- Evaluation of generated text

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Logits	raw scores before softmax	temperature modifies them	vocab scores	Not probabilities yet
Low temperature	sharper distribution	deterministic/factual	T=0.1	Good for extraction
High temperature	flatter distribution	creative/diverse	T=1.2	More hallucination risk
Greedy decoding	choose max token	no randomness	argmax	Often T=0 behavior

Calibration	probability quality	affects confidence	uncertain output	Not solved by temperature alone
-------------	---------------------	--------------------	------------------	---------------------------------

5. Algorithm / Working Process

1. Model produces logits for next token.
2. Divide logits by temperature T .
3. Apply softmax.
4. Sample from resulting distribution.
5. Repeat for each generated token.

6. Mathematical Foundation

$$P(\text{token}_i) = \exp(\text{logit}_i / T) / \sum_j \exp(\text{logit}_j / T)$$

Effects:

```
T < 1 -> sharper distribution  
T = 1 -> original distribution  
T > 1 -> flatter distribution  
T -> 0 -> argmax-like
```

7. Practical Implementation

```
import torch  
  
logits = torch.tensor([2.0, 1.0, 0.1])  
  
for temperature in [0.2, 1.0, 2.0]:  
    probs = torch.softmax(logits / temperature, dim=-1)  
    print(temperature, probs.tolist())
```

8. Code Explanation

Dividing logits by a small temperature increases score differences, making the top token dominate. A larger temperature reduces score differences, spreading probability across more tokens.

9. Training / Evaluation

Temperature is used at inference, not training. Evaluate by task: exactness for extraction/code, factuality for QA, diversity for creative writing, human preference for chat.

10. Complexity and Cost

Temperature adds negligible compute. Indirect cost can rise if high temperature causes longer, less useful, or retry-prone outputs.

11. Common Use Cases

- Low temperature for extraction, classification, math, code edits
- Medium temperature for helpful chat
- Higher temperature for brainstorming, creative writing, diverse samples

12. Common Mistakes

- Using high temperature for factual QA
- Expecting temperature to fix bad prompts
- Setting temperature high with no output validation
- Confusing temperature with top-p/top-k
- Comparing outputs without fixed seeds/settings

13. Edge Cases / Limitations

Temperature cannot add knowledge or guarantee correctness. Very high temperature can produce incoherent text. Very low temperature can repeat or get stuck.

14. Variations

Variation	What changes	When to use	Placement value
Greedy	always argmax	deterministic tasks	Important
Temperature sampling	scales logits	diversity control	Very important
Annealed temperature	changes over generation	advanced decoding	Research
Combined with top-p	filters and controls randomness	production LLMs	Very important

15. Related Topics

Temperature connects to top-k sampling, top-p sampling, decoder-only models, prompting, and hallucination control.

16. Interview Questions

1. What is temperature?
A parameter that scales logits before sampling.
2. Formula?
`softmax(logits / T)`.
3. What does low temperature do?
Makes distribution sharper and output more deterministic.
4. What does high temperature do?
Increases randomness and diversity.
5. Is temperature used during training?
Usually no; it is a decoding-time parameter.
6. Best temperature for factual extraction?
Low, often near zero.
7. Can temperature prevent hallucination?
It can reduce randomness but not guarantee truth.

8. Temperature vs top-p?
Temperature reshapes probabilities; top-p filters cumulative probability mass.
9. What happens as $T \rightarrow 0$?
Decoding becomes argmax-like.
10. Why use higher temperature?
To generate diverse or creative outputs.

17. Practice Tasks

- Plot probability distributions for several temperatures.
- Generate five answers with low and high temperature.
- Test factual QA accuracy at different temperatures.
- Combine temperature with top-p.
- Build a decoding parameter playground.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Sampling Visualizer	Shows probability changes	Python, Streamlit	Toy logits	Great interview demo
Creative Prompt Lab	Compares diversity settings	LLM API	Writing prompts	Practical prompting
QA Stability Tester	Measures answer variance	Python	QA set	Evaluation skill

19. Quick Revision

- Key idea: controls randomness by scaling logits.
- Main formula: $P_i = \text{softmax}(z_i/T)$.
- When to use: generation decoding.
- Important metrics: factuality, diversity, validity.
- Common traps: high temp for exact tasks.
- Interview one-liner: temperature decides how sharply the model follows its top token preferences.

20. Final Cheat Sheet

Item	Summary
Definition	Logit scaling for sampling
Input/output	logits -> adjusted probabilities
Main steps	divide logits, softmax, sample
Hyperparameters	temperature value
Metrics	diversity, accuracy, validity
Pros	easy creativity control

Item	Summary
Cons	no correctness guarantee
Best use cases	generation style control

Top-k Sampling

1. Overview

Top-k sampling restricts next-token sampling to the k most probable tokens. It prevents the model from sampling extremely unlikely tokens while preserving some randomness.

2. Intuition

Instead of choosing from the entire vocabulary, top-k says: "Only choose from the best k candidates."

3. Prerequisites

- Logits and probabilities
- Softmax
- Sampling
- Decoder-only generation
- Temperature

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
k	number of allowed tokens	controls candidate set	$k=50$	Fixed cutoff
Filtering	remove low-ranked tokens	avoids nonsense	set to $-\infty$	Before softmax
Renormalization	probabilities sum to 1	valid sampling	top tokens only	Required
Diversity	random among candidates	less boring than greedy	creative output	Quality trade-off
Fixed size	always k tokens	simple but rigid	even if distribution sharp	Top-k vs top-p

5. Algorithm / Working Process

1. Compute logits.
2. Keep only top k logits.
3. Set all other logits to $-\infty$.
4. Apply softmax to remaining logits.

5. Sample next token.

6. Mathematical Foundation

Let S_k be the set of top k tokens by probability:

$$P'(i) = P(i) / \sum_{j \in S_k} P(j), \text{ if } i \in S_k$$
$$P'(i) = 0, \text{ otherwise}$$

7. Practical Implementation

```
import torch

def top_k_sample(logits, k):
    values, indices = torch.topk(logits, k)
    probs = torch.softmax(values, dim=-1)
    choice = torch.multinomial(probs, num_samples=1)
    return indices[choice]

logits = torch.tensor([3.0, 2.5, 1.0, 0.2, -1.0])
print(top_k_sample(logits, k=3).item())
```

8. Code Explanation

`torch.topk` selects the top k scores. Softmax renormalizes only those scores. `multinomial` samples one token from the filtered distribution.

9. Training / Evaluation

Top-k is an inference-time decoding method. Evaluate generated text for coherence, diversity, factuality, repetition, and task success. For code, use `pass@k` or test success.

10. Complexity and Cost

Top-k adds sorting/selection cost over vocabulary. Efficient implementations use partial top-k selection. Compared with model forward pass, decoding filter cost is usually small.

11. Common Use Cases

- Creative generation
- Chat responses
- Story writing
- Candidate answer generation
- Code suggestions

12. Common Mistakes

- Setting **k** too high, allowing poor tokens
- Setting **k** too low, making output repetitive
- Forgetting renormalization
- Confusing **k** candidates with generated token count
- Using top-k where top-p adapts better

13. Edge Cases / Limitations

Top-k is rigid. If the distribution is very confident, **k=50** may include bad choices. If the distribution is flat, **k=5** may remove useful options.

14. Variations

Variation	What changes	When to use	Placement value
Greedy	k=1	deterministic tasks	Important
Top-k	fixed candidate count	simple diversity	Very important
Top-p	dynamic candidate count	adaptive sampling	Very important
Beam search	keeps best sequences	translation/summarization	Important

15. Related Topics

Top-k connects to temperature, top-p, decoder-only generation, logits, and prompt evaluation.

16. Interview Questions

1. What is top-k sampling?
Sampling only from the **k** highest-probability tokens.
2. Why use it?
To avoid very unlikely tokens while keeping diversity.
3. What happens when **k=1**?
Greedy decoding.
4. Does top-k change model weights?
No, it is inference-time decoding.
5. What is renormalization?
Rescaling remaining probabilities to sum to 1.
6. Top-k vs temperature?
Top-k filters candidates; temperature reshapes probabilities.
7. Top-k vs top-p?
Top-k uses fixed count; top-p uses cumulative probability mass.
8. Risk of high **k**?
Low-quality tokens may remain.
9. Risk of low **k**?
Repetitive or dull output.
10. Is top-k good for exact extraction?
Usually greedy/low randomness is better.

17. Practice Tasks

- Implement top-k filtering.
- Compare generations for $k=1$, 10, and 50.
- Combine top-k with temperature.
- Measure output diversity.
- Debug sampling when probabilities do not sum to 1.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Decoding Playground	Tests top-k values	Streamlit, PyTorch	Toy prompts	LLM fundamentals
Story Sampler	Generates varied stories	HF Transformers	Writing prompts	Generative AI demo
Code Candidate Generator	Samples multiple code completions	Transformers	HumanEval-style tasks	Evaluation relevance

19. Quick Revision

- Key idea: sample from top k tokens only.
- Main formula: renormalize probabilities over S_k .
- When to use: controlled diversity.
- Important metrics: diversity, coherence, success rate.
- Common traps: confusing k with output length.
- Interview one-liner: top-k cuts the vocabulary down to the most likely candidates before sampling.

20. Final Cheat Sheet

Item	Summary
Definition	Fixed-size candidate sampling
Input/output	logits -> sampled token
Main steps	top-k, mask, softmax, sample
Hyperparameters	k , temperature
Metrics	diversity, validity, success
Pros	simple, avoids tail tokens
Cons	rigid cutoff
Best use cases	creative but controlled generation

Top-p Sampling

1. Overview

Top-p sampling, or nucleus sampling, samples from the smallest set of tokens whose cumulative probability is at least p . Unlike top-k, the candidate set size changes depending on model confidence.

2. Intuition

Top-p says: "Choose from the smallest group of likely words that together cover, say, 90% probability." If the model is confident, the group is small. If uncertain, the group grows.

3. Prerequisites

- Probability distributions
- Sorting
- Cumulative sums
- Sampling
- Temperature and top-k

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Nucleus	dynamic token set	adapts to confidence	tokens covering 0.9 mass	Top-p vs top-k
p	cumulative probability threshold	controls diversity	$p=0.9$	Not token count
Sorting	rank by probability	needed for cumulative mass	high to low	Implementation
Tail removal	removes low-probability tail	improves coherence	rare tokens removed	Hallucination risk
Dynamic size	candidate count varies	more adaptive	3 or 100 tokens	Advantage

5. Algorithm / Working Process

1. Compute logits.
2. Apply temperature if used.
3. Convert logits to probabilities.
4. Sort tokens by probability descending.
5. Keep smallest prefix whose cumulative probability $\geq p$.
6. Renormalize and sample.

6. Mathematical Foundation

Find smallest set S_p :

$$S_p = \text{smallest sorted set such that } \sum_{i \in S_p} P(i) \geq p$$

Renormalized distribution:

$$P'(i) = P(i) / \sum_{j \in S_p} P(j), \text{ if } i \in S_p$$

$$P'(i) = 0, \text{ otherwise}$$

7. Practical Implementation

```
import torch

def top_p_sample(logits, p=0.9):
    probs = torch.softmax(logits, dim=-1)
    sorted_probs, sorted_indices = torch.sort(probs, descending=True)
    cumulative = torch.cumsum(sorted_probs, dim=-1)

    keep = cumulative <= p
    keep[0] = True # always keep at least one token
    filtered_probs = sorted_probs[keep]
    filtered_indices = sorted_indices[keep]

    filtered_probs = filtered_probs / filtered_probs.sum()
    choice = torch.multinomial(filtered_probs, 1)
    return filtered_indices[choice]

logits = torch.tensor([4.0, 2.0, 1.0, 0.5, -1.0])
print(top_p_sample(logits, p=0.9).item())
```

8. Code Explanation

The code sorts tokens by probability, computes cumulative mass, keeps the likely nucleus, renormalizes, and samples. The first token is always kept so sampling remains valid.

9. Training / Evaluation

Top-p is decoding-time only. Evaluate diversity, coherence, factuality, repetition, human preference, and downstream task success. For production, test multiple prompts and seeds.

10. Complexity and Cost

Top-p requires sorting vocabulary probabilities, which adds overhead. The model forward pass is usually still the expensive part.

11. Common Use Cases

- Chat assistants
- Creative writing
- Brainstorming
- Synthetic data generation
- Open-ended QA

12. Common Mistakes

- Thinking p means number of tokens
- Setting p too high for factual tasks
- Combining high temperature and high top-p carelessly
- Forgetting to keep at least one token
- Not validating outputs

13. Edge Cases / Limitations

Top-p can still sample wrong or hallucinated tokens. Very low p can become greedy-like. Very high p includes more low-probability tail tokens.

14. Variations

Variation	What changes	When to use	Placement value
Top-p	probability mass cutoff	adaptive generation	Very important
Top-k	fixed token count	simple candidate control	Very important
Typical sampling	keeps tokens with typical surprise	advanced decoding	Research
Beam search	sequence-level search	translation/summarization	Important

15. Related Topics

Top-p connects to temperature, top-k, prompting, decoder-only models, hallucination, and evaluation.

16. Interview Questions

1. What is top-p sampling?
Sampling from the smallest token set whose cumulative probability reaches p .
2. Why is it called nucleus sampling?
It samples from the high-probability nucleus of the distribution.
3. Top-p vs top-k?
Top-p dynamic mass cutoff; top-k fixed count.
4. What does $p=0.9$ mean?
Keep tokens covering at least 90% probability mass.
5. Does top-p train the model?
No, it only affects decoding.
6. Why renormalize?
Remaining probabilities must sum to 1.

7. Risk of high top-p?
More unlikely tokens can be sampled.
8. Risk of low top-p?
Less diversity and possible repetition.
9. How combine with temperature?
Temperature reshapes probabilities before/alongside filtering.
10. Best for factual tasks?
Use low randomness; top-p may be set lower or sampling disabled.

17. Practice Tasks

- Implement top-p sampling.
- Compare top-k and top-p candidate sizes.
- Plot nucleus size across different logits.
- Generate outputs with $p=0.5, 0.9, 0.98$.
- Build tests ensuring probability sums to 1.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Nucleus Sampler Demo	Visualizes dynamic token sets	Python, Streamlit	Toy logits	Clear decoding knowledge
Prompt Diversity Lab	Measures output variation	LLM API	Prompt suite	AI eval signal
Synthetic Data Sampler	Tunes top-p for data quality	HF/LLM API	Classification schemas	Practical gen-AI

19. Quick Revision

- Key idea: sample from tokens covering probability mass p .
- Main formula: $\sum P(i) \geq p$.
- When to use: adaptive open-ended generation.
- Important metrics: coherence, diversity, factuality.
- Common traps: treating p as token count.
- Interview one-liner: top-p keeps the likely nucleus of tokens and samples from it.

20. Final Cheat Sheet

Item	Summary
Definition	Cumulative-probability sampling
Input/output	logits -> sampled token
Main steps	sort, cumulative sum, keep nucleus, sample
Hyperparameters	p , temperature

Item	Summary
Metrics	diversity, coherence, validity
Pros	adaptive candidate set
Cons	can still hallucinate
Best use cases	open-ended generation

Instruction Tuning

1. Overview

Instruction tuning fine-tunes a pretrained language model on instruction-response examples so it learns to follow user requests. It is a key step that turns a base language model into a useful assistant. It is used in chatbots, coding assistants, enterprise copilots, tool-using agents, and safety-aligned systems.

2. Intuition

A base model predicts likely text. An instruction-tuned model learns the pattern: user asks a task, assistant gives a helpful answer. It is the difference between autocomplete and assistant behavior.

3. Prerequisites

- Pretraining
- Fine-tuning
- Prompt-response datasets
- Supervised learning
- Evaluation of generated outputs

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Instruction dataset	prompts + ideal responses	teaches following	"Summarize..." -> answer	Data quality
SFT	supervised fine-tuning	common tuning stage	train on responses	Loss masking
Chat template	formats roles	model learns dialogue	system/user/assistant	Token format matters
Preference tuning	learns preferred responses	alignment	RLHF/DPO	After SFT
Safety behavior	refusal/helpfulness rules	deployment quality	harmful request handling	Alignment

5. Algorithm / Working Process

1. Start with pretrained base model.
2. Collect instruction-response pairs.
3. Format examples with the model's chat template.
4. Train model to predict assistant response tokens.
5. Validate helpfulness, correctness, and format.
6. Optionally apply preference tuning such as RLHF or DPO.
7. Deploy with system prompts and safety checks.

6. Mathematical Foundation

Supervised instruction tuning loss:

$$L_{\text{SFT}} = - \sum_{t \text{ in assistant response}} \log P(y_t \mid \text{instruction}, y_{<t})$$

Often, loss is masked on user prompt tokens so the model is optimized mainly for assistant responses.

Preference tuning idea:

preferred response should receive higher score/probability than rejected response

DPO-style objective conceptually increases:

$$\log P(y_{\text{chosen}} \mid x) - \log P(y_{\text{rejected}} \mid x)$$

7. Practical Implementation

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("HuggingFaceH4/zephyr-7b-beta")

messages = [
    {"role": "system", "content": "You are a concise ML interview mentor."},
    {"role": "user", "content": "Explain self-attention in two sentences."},
    {"role": "assistant", "content": "Self-attention lets each token weigh other visible tokens. It computes softmax(QK^T/sqrt(d_k))V to build context-aware representations."},
]

formatted = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=False,
)
```

```
print(formatted)
```

8. Code Explanation

Instruction-tuned chat models expect a specific role format. `apply_chat_template` formats system, user, and assistant messages exactly as the model was trained to read them.

9. Training / Evaluation

Evaluate with held-out instruction sets, human preference, factuality checks, format adherence, safety tests, and task-specific benchmarks. Watch for overfitting to response style, verbosity, refusal errors, and benchmark contamination.

10. Complexity and Cost

Instruction tuning is cheaper than pretraining but can still be expensive for large models. PEFT methods such as LoRA/QLoRA reduce memory. Data quality matters more than raw dataset size.

11. Common Use Cases

- Chat assistants
- Coding copilots
- Customer support bots
- Tool-using agents
- Structured extraction assistants
- Domain expert assistants

12. Common Mistakes

- Training on low-quality synthetic answers without filtering
- Not masking prompt tokens when intended
- Using wrong chat template
- Confusing instruction tuning with prompting
- Ignoring safety and refusal evaluation
- Over-tuning until general ability drops

13. Edge Cases / Limitations

Instruction tuning improves following instructions but does not guarantee truth. It can make models overly agreeable, verbose, or refusal-prone. It may not add reliable new knowledge.

14. Variations

Variation	What changes	When to use	Placement value
SFT	train on prompt-answer pairs	base assistant behavior	Very important
Multi-turn tuning	dialogue examples	chat assistants	Important

Variation	What changes	When to use	Placement value
Tool instruction tuning	tool call traces	agents	AI engineering
RLHF	reward model + RL	preference alignment	Important concept
DPO	direct preference optimization	simpler preference tuning	Very important

15. Related Topics

Instruction tuning connects pretraining, fine-tuning, prompting, preference alignment, RLHF, DPO, safety evaluation, and chat templates.

16. Interview Questions

1. What is instruction tuning?
Fine-tuning a model on instruction-response examples.
2. Why is it needed?
Base LMs predict text; instruction tuning teaches task-following behavior.
3. SFT meaning?
Supervised fine-tuning.
4. What is a chat template?
The role/message formatting expected by a chat model.
5. Why mask user prompt loss?
To train the model mainly on assistant response behavior.
6. Instruction tuning vs prompting?
Tuning changes weights; prompting changes input.
7. Instruction tuning vs pretraining?
Pretraining learns broad language; instruction tuning aligns behavior to requests.
8. What is RLHF?
Reinforcement learning from human feedback for preference alignment.
9. What is DPO?
A preference tuning method that directly optimizes chosen over rejected responses.
10. Does instruction tuning guarantee factuality?
No, retrieval/evaluation may still be needed.

17. Practice Tasks

- Format examples with a chat template.
- Build a tiny instruction dataset.
- Fine-tune a small model with LoRA for response style.
- Evaluate format-following accuracy.
- Compare base vs instruction-tuned model outputs.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
ML Interview Tutor	Instruction-tuned mentor responses	PEFT, HF	Custom Q&A	Direct placement value
Tool-Calling Assistant Dataset	Creates tool-use examples	Python, JSONL	Synthetic tasks	Agent engineering
Preference Pair Evaluator	Compares chosen/rejected answers	DPO/TRL	HH-RLHF style data	Alignment knowledge

19. Quick Revision

- Key idea: teach a base model to follow instructions.
- Main formula: response-token cross-entropy.
- When to use: assistant behavior and task formatting.
- Important metrics: helpfulness, correctness, format adherence, safety.
- Common traps: wrong chat template, low-quality data.
- Interview one-liner: instruction tuning turns a language model into an instruction-following assistant.

20. Final Cheat Sheet

Item	Summary
Definition	Fine-tuning on instruction-response data
Input/output	instruction dataset -> assistant model
Main steps	format, mask, train, validate, align
Hyperparameters	LR, epochs, LoRA rank, max length
Metrics	helpfulness, win rate, safety, format validity
Pros	better task following
Cons	data quality sensitive, no truth guarantee
Best use cases	chatbots, copilots, agents